



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Introduction

1.3 Graph Theory

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

1. Introduction

- General aspects
- Fundamentals of networking
- **Graph theory**

2. The protocol stack

3. Impact on programming of on-line games

4. Other stories

Remember from last class



CITM

- The Internet is a large network with many nodes, links, topologies and architectures.
- It is unreliable and adds latency, jitter, and packet losses. Packets can even arrive out of order!
- These will cause lag and glitches in your game unless we remedy it.

- The Internet is a large network with many nodes, links, topologies and architectures.
- It is unreliable and adds latency, jitter, and packet losses. Packets can even arrive out of order!
- These will cause lag and glitches in your game unless we remedy it.
- **Today, we will see the theoretical foundation behind the Internet and its algorithms.**



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Graphs:

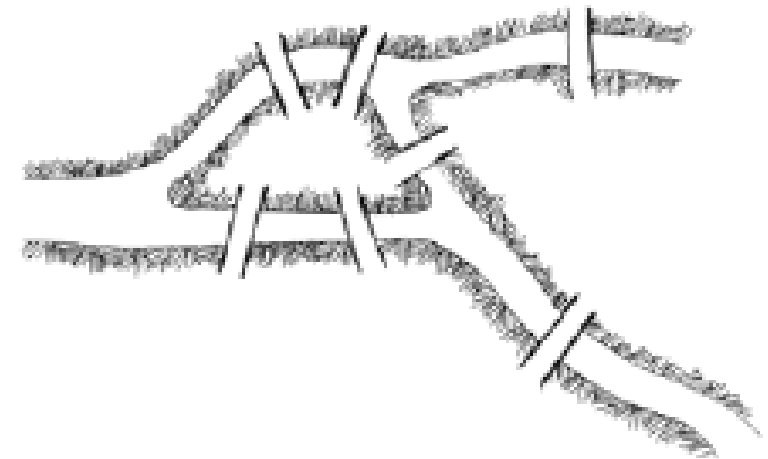
A short preamble

The seven bridges of Königsberg



CITM

- In the city of Königsberg (Kaliningrad nowadays) apparently the citizens wondered if it was possible to cross the seven bridges connecting the two islands within the Pregolya river in a way that each bridge was crossed only once.



The seven bridges of Königsberg (II)



CITM

- Meet Leonhard Euler, famous mathematician known for:
 - The mathematical constant “e”
 - The Euler’s formula
 - The Euler’s theorem
 - Solving the problem of the “seven bridges”

Leonhard Euler

Basilea, 15th April 1707

Saint Petersburg, 18th September 1783



The seven bridges of Königsberg (III)

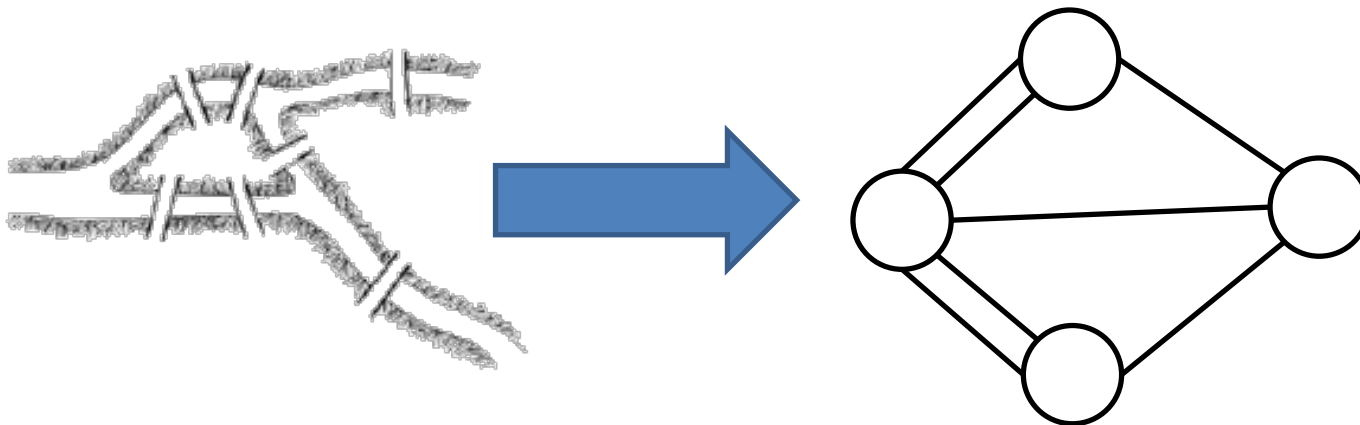
- It was clear that you could not cross the seven bridges, but the idea was to demonstrate it formally.
- Euler did it in 1736 and published it in a scientific journal with the name: *Solutio problematis ad geometriam situs pertinentis*.

<http://eulerarchive.maa.org>



The seven bridges of Königsberg (IV)

*It was the first formal
declaration of a graph and
its application as a
mathematical tool*





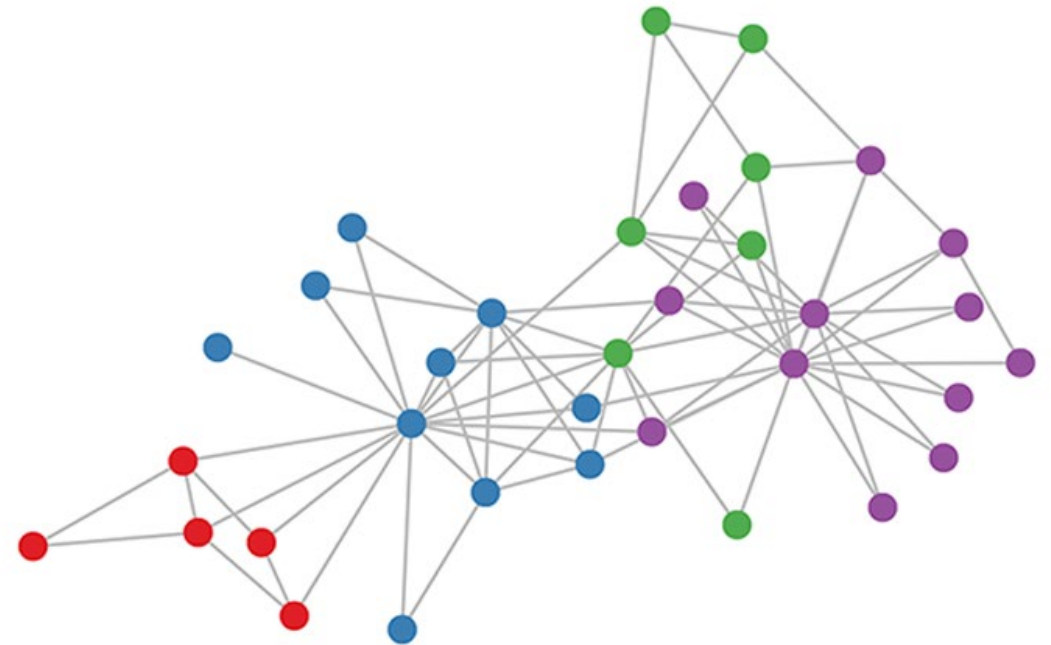
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Graphs: Concepts and applications

What is a graph?

- **A graph is a set of objects partially or fully connected together**
 - The objects are called vertices; what we understand as nodes
 - The connections are called edges; what we understand as links



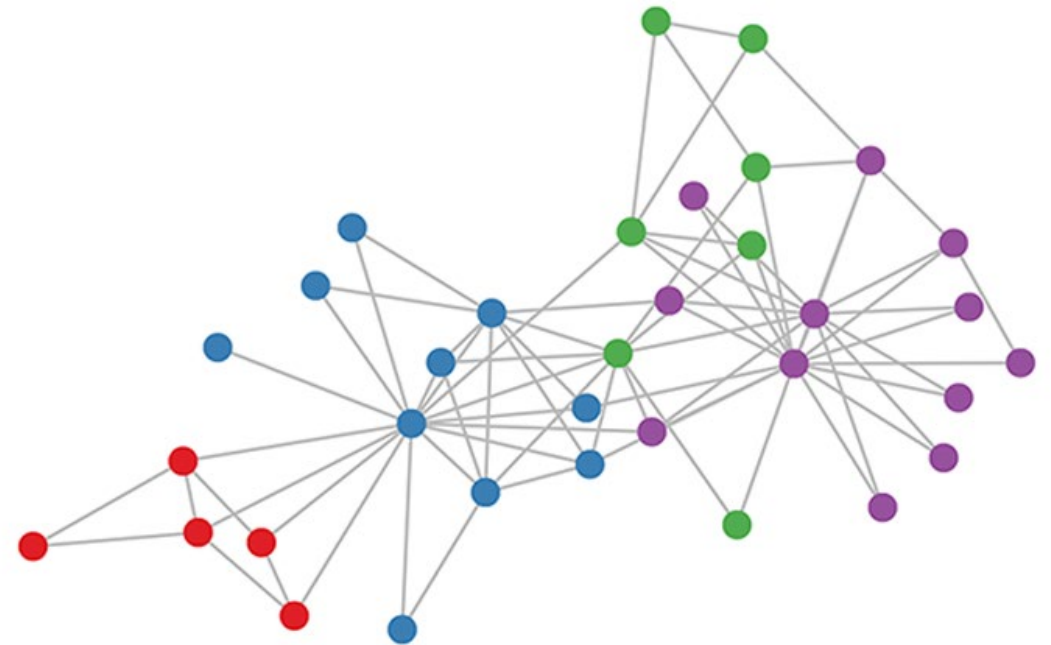
What is a graph?

- **A graph is a mathematical abstraction describing relations between entities**

Graph $G = (V, E)$

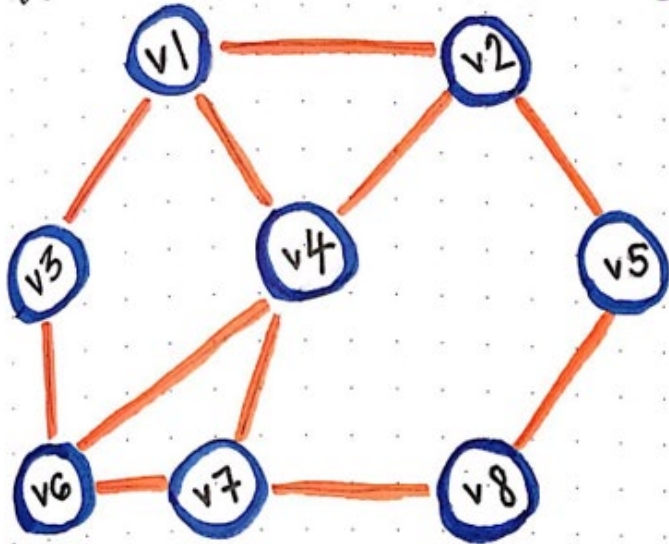
Set of vertices V

Set of edges E



Mathematical Notation of a Graph

(Formally) Defining a Graph



8 vertices/nodes
11 edges/links

$$G = (V, E)$$

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$$

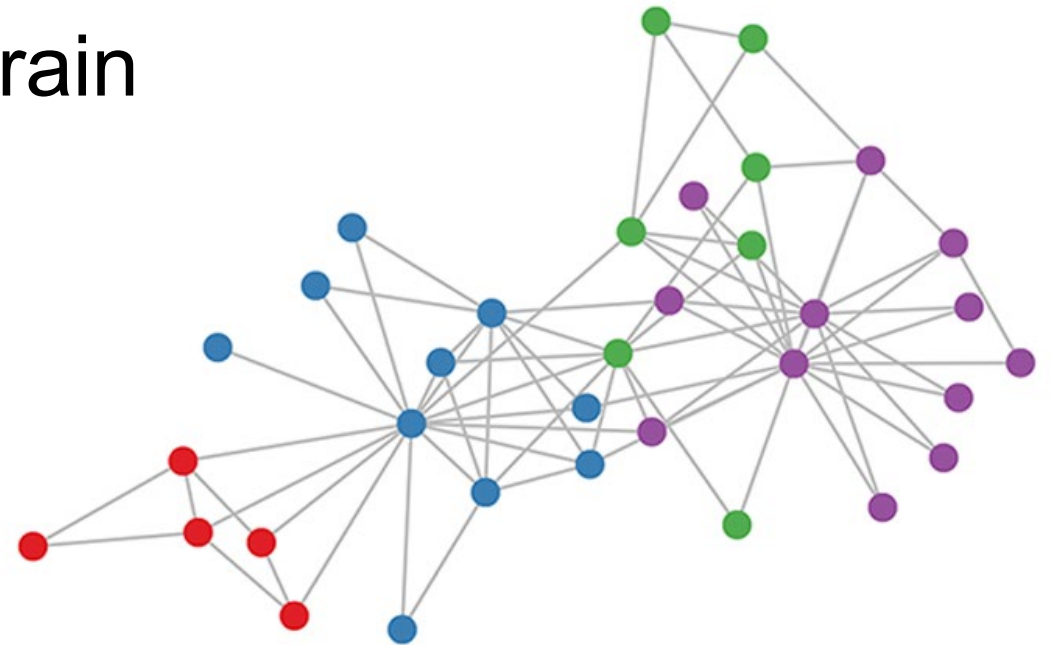
$$E = \{ \{v_1, v_2\}, \{v_1, v_3\}, \{v_1, v_4\}, \{v_2, v_4\}, \{v_2, v_5\}, \{v_3, v_6\}, \{v_4, v_6\}, \{v_4, v_7\}, \{v_5, v_8\}, \{v_6, v_7\}, \{v_7, v_8\} \}$$

these edge definitions are unordered pairs!

- $G = (V, E)$ is the formal mathematical notation for defining graphs.
- A graph G is an ordered pair of a set V vertices and E , a set of edges.
- An ordered pair is a pair of mathematical objects in which the order of objects in the pair matters.

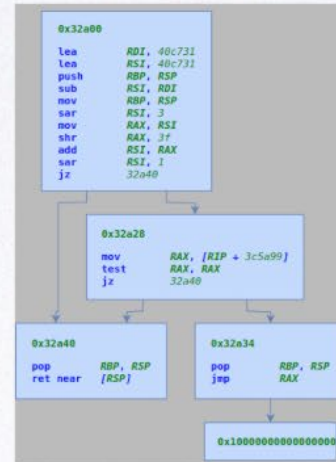
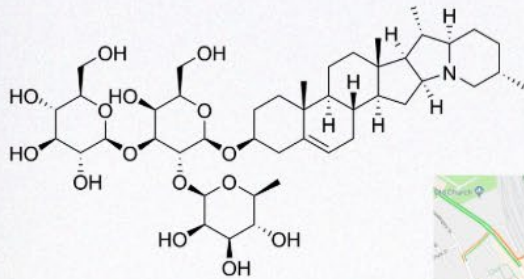
Examples of graphs

1. All the websites in the Internet
2. Roads in a city
3. Human acquaintances (six degrees of separation)
4. Grammar – Words in a language
5. Neurons and synapses in the brain
6. Molecules in an element
7. Interaction between species
8. Any communication network

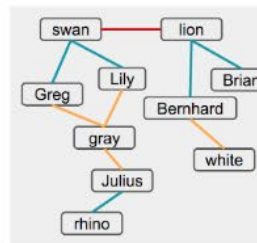


Examples of graphs

Graph Structured Data Comes in Many Forms



1. Lily is a swan.
2. Bernhard is a lion.
3. Greg is a swan.
4. Bernhard is white.
5. Brian is a lion.
6. Lily is gray.
7. Julius is a rhino.
8. Julius is gray.
9. Greg is gray.
10. Swan is afraid of lion.



Q: Is Greg afraid of Bernhard?
A: Yes.

Q: Do Greg and Lily have the same color?
A: Yes.

Q: Is Julius a lion?
A: No.

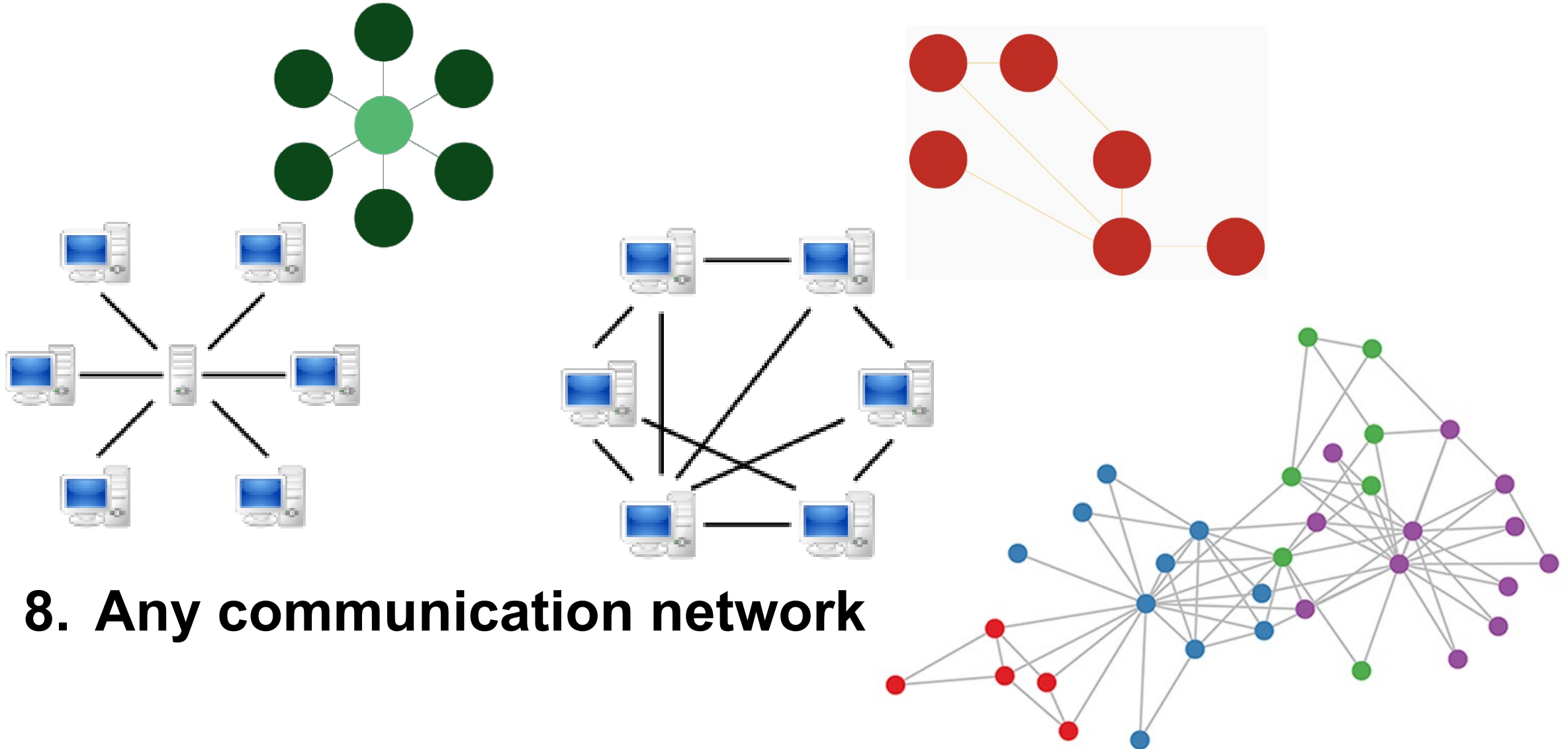
• • •

Infer graph

Use graph

Source:
DeepMind
(Google)

Examples of graphs



8. Any communication network



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

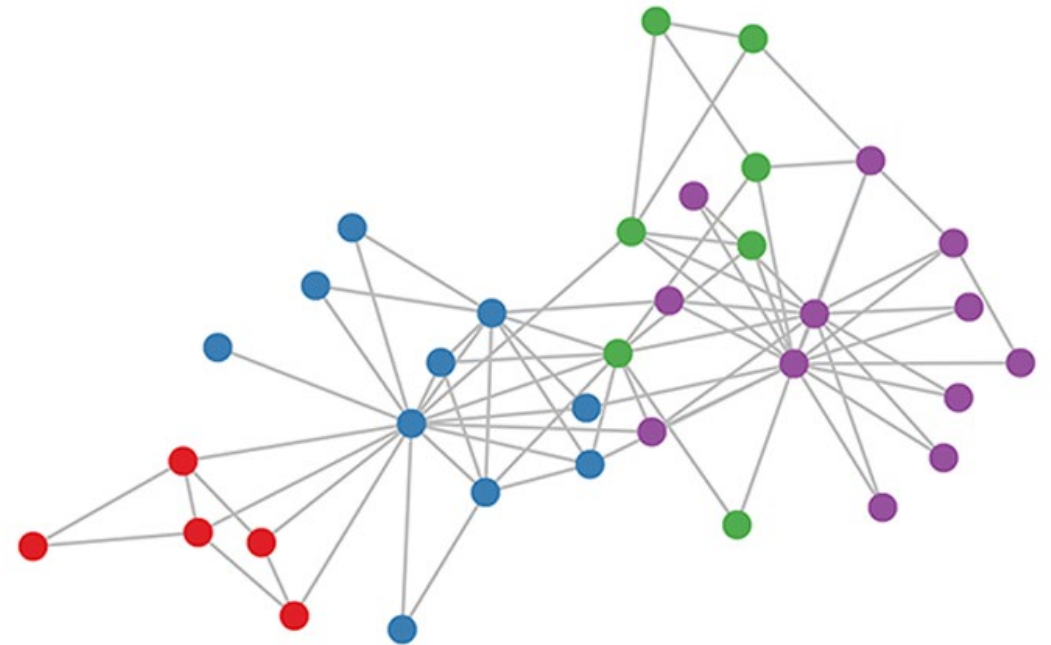
Characteristics and types

Characteristics of a graph



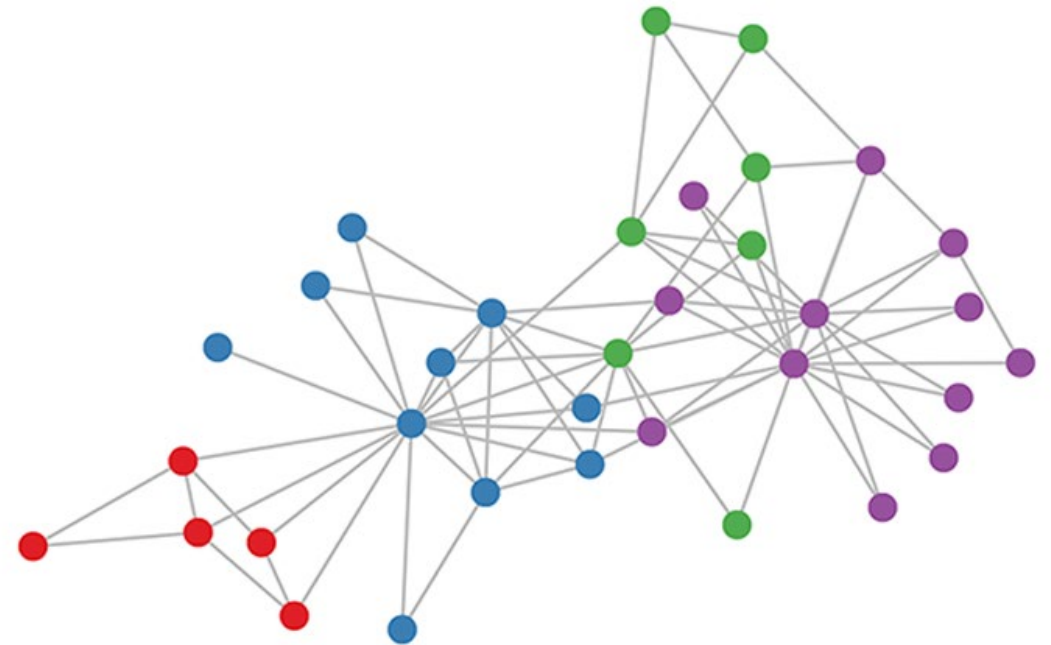
CITM

- **Degree:** the degree of a vertex is the number of edges that arrive or stem from it. The higher is the value, the more connected is the vertex.
- Density
- Diameter
- Etc...



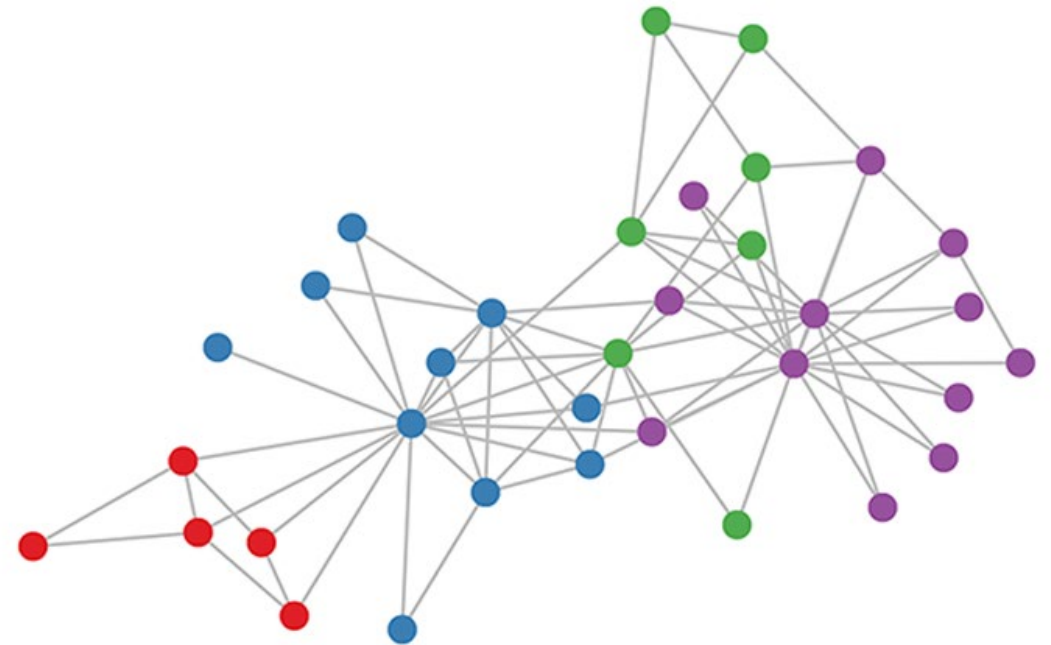
Characteristics of a graph

- Degree
- **Density:** the density of a graph is the ratio of number of edges to the maximum possible number of edges. Graphs are dense (high ratio) or sparse (low ratio)
- Diameter
- Etc...

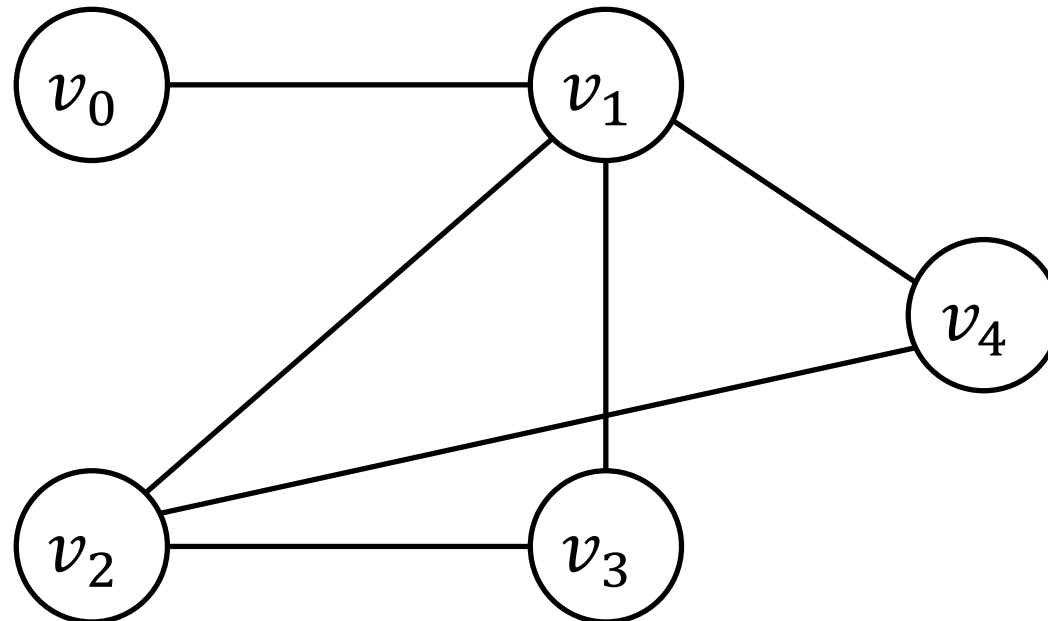


Characteristics of a graph

- Degree
- Density
- **Diameter:** maximum distance (hops) between any two nodes in the graph
- Etc...



What is the density and diameter of the following graph? Which is the degree of its nodes?



What is the density and diameter of the following graph? Which is the degree of its nodes?

- Number of possible edges (combinatorics) is

$$|E|_{max} = \frac{|V|(|V| - 1)}{2} = 10$$



What is the density and diameter of the following graph? Which is the degree of its nodes?

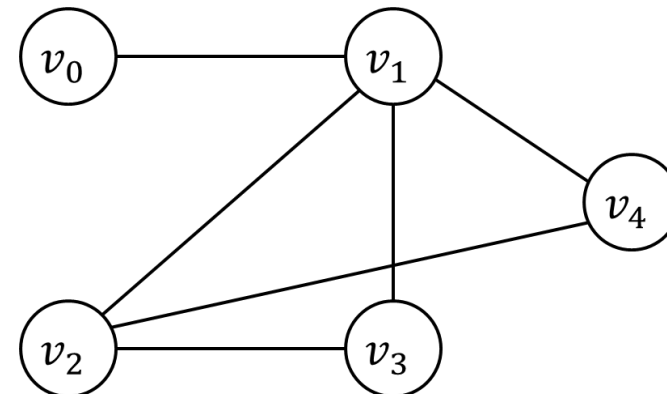
- The density is, thus,

$$D = \frac{|E|}{|E|_{max}} = \frac{6}{10} = 0.6$$



What is the density and diameter of the following graph? Which is the degree of its nodes?

- The diameter is 2, basically the distance to go from v_0 to any other vertex except v_1 .



What is the density and diameter of the following graph? Which is the degree of its nodes?

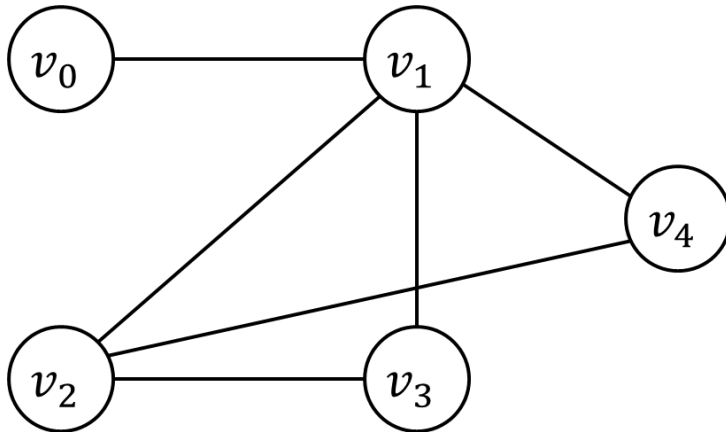
- The degree of the nodes is
 - $\text{Deg}(v_0) = 1$
 - $\text{Deg}(v_1) = 4$
 - $\text{Deg}(v_2) = 3$
 - $\text{Deg}(v_3) = 2$
 - $\text{Deg}(v_4) = 2$



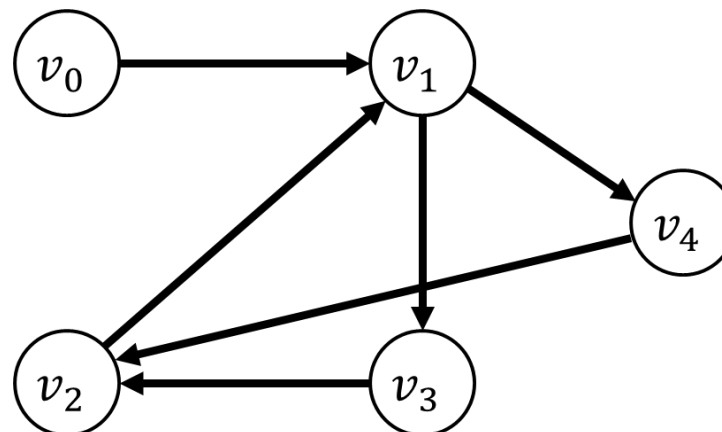
Graph types

- The main distinction between types of graphs comes from the type of edges

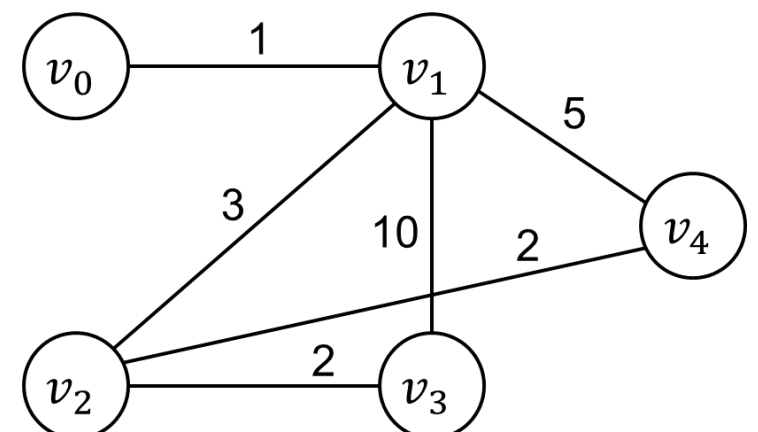
Normal graph
(bidirectional edges)



Directional graph
(unidirectional edges)

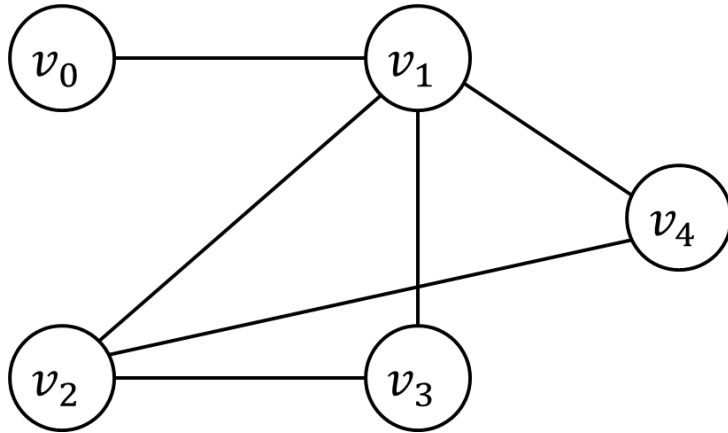


Weighted graph
(non-unit edges)

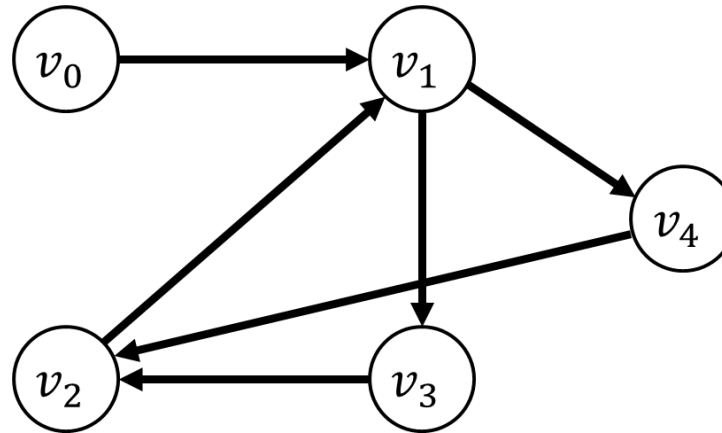


Graph types

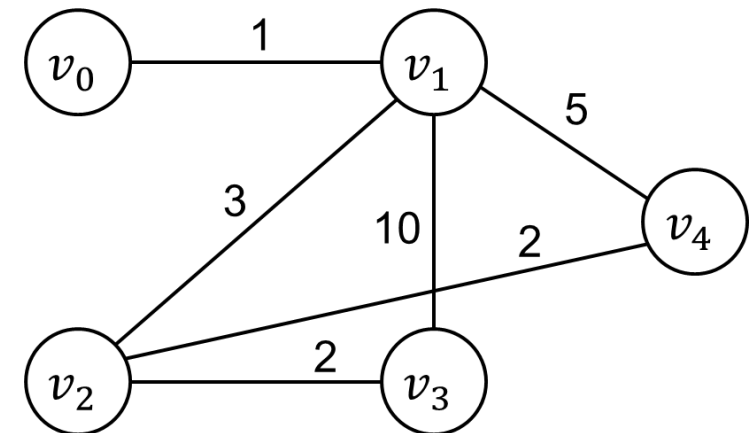
Normal graph
(bidirectional edges)



Directional graph
(unidirectional edges)



Weighted graph
(non-unit edges)



Which type of graph you think best represents a network like the Internet?

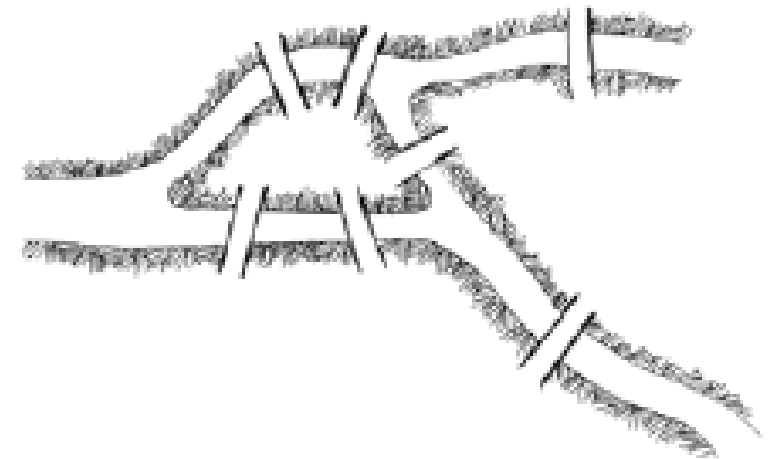


The seven bridges of Königsberg



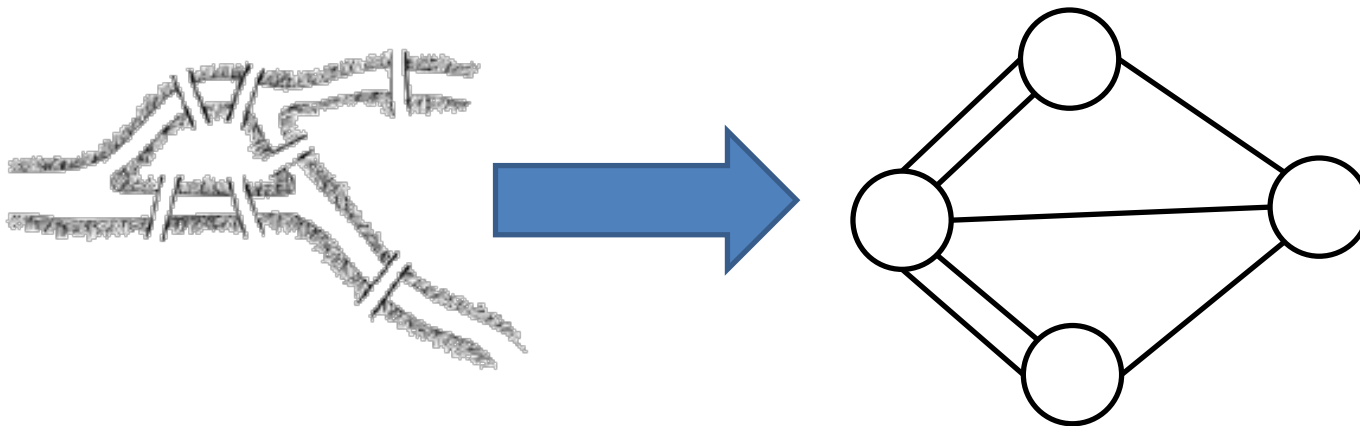
CITM

- In the city of Königsberg (Kaliningrad nowadays) apparently the citizens wondered if it was possible to cross the seven bridges connecting the two islands within the Pregolya river in a way that each bridge was crossed only once.



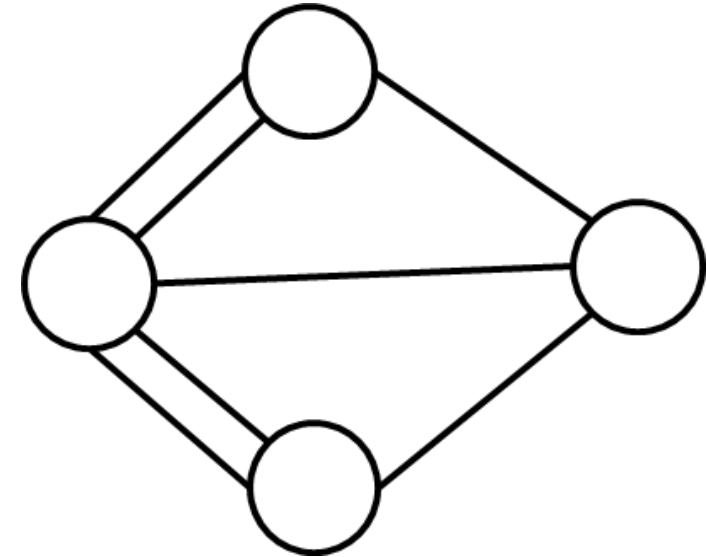
The seven bridges of Königsberg

*It was the first formal
declaration of a graph and
its application as a
mathematical tool*



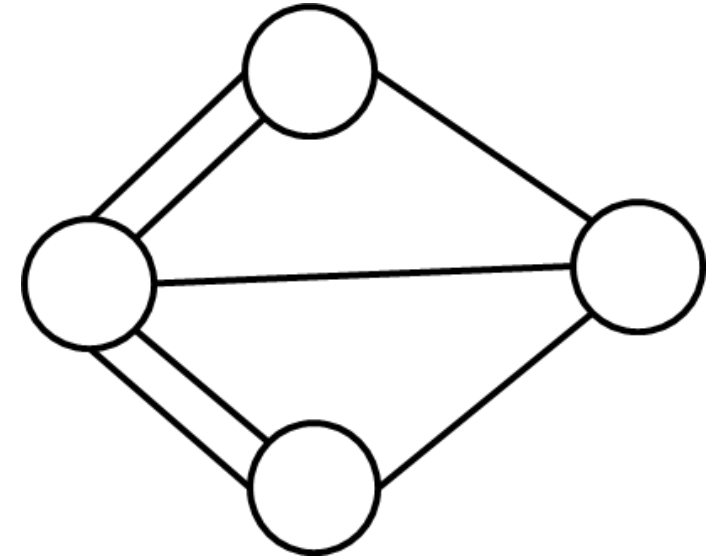
The seven bridges of Königsberg (solved)

- What Euler observed is that, using the graph, a circuit was only possible if there was exactly two nodes (or none) with an odd degree.
- What happens in our problem?
 - Which is the degree of the vertices?
 - Can we make a circuit?



The seven bridges of Königsberg (solved)

- What Euler observed is that, using the graph, a circuit was only possible if there was exactly two nodes (or none) with an odd degree.
- Therefore, in this case, the circuit is impossible to realize.
 - One node has a degree of 5.
 - Three nodes have a degree of 3.



Generalization of Eulerian graph

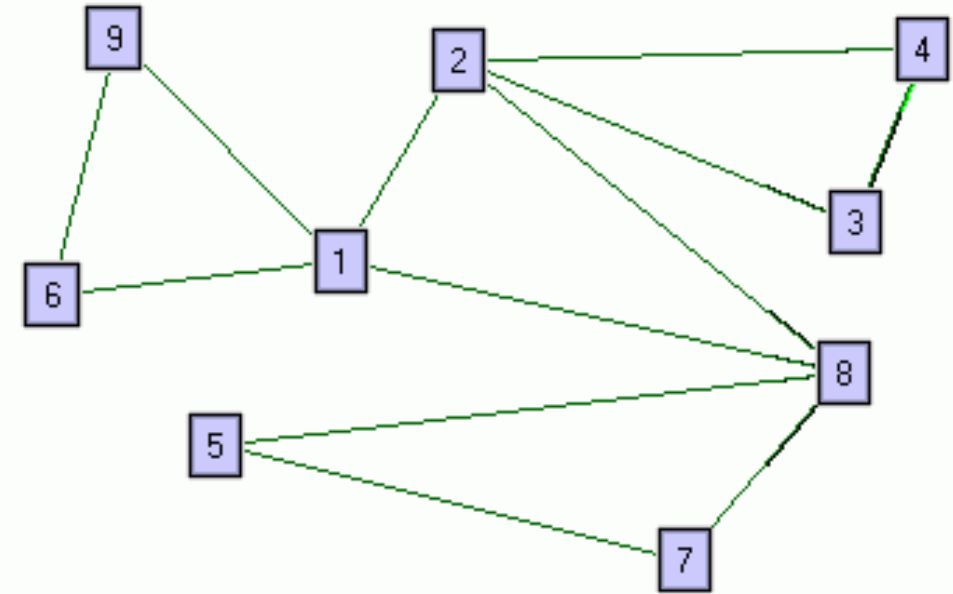
- A graph is Eulerian $\rightarrow$ has an Eulerian path $\rightarrow$ has a path that goes through all edges only once $\rightarrow$ only if it has two nodes (or none) with odd degree.
- In case there is no node with odd degree, then there exists a path that goes through all the links only once and that goes back to the starting point.



Exercise

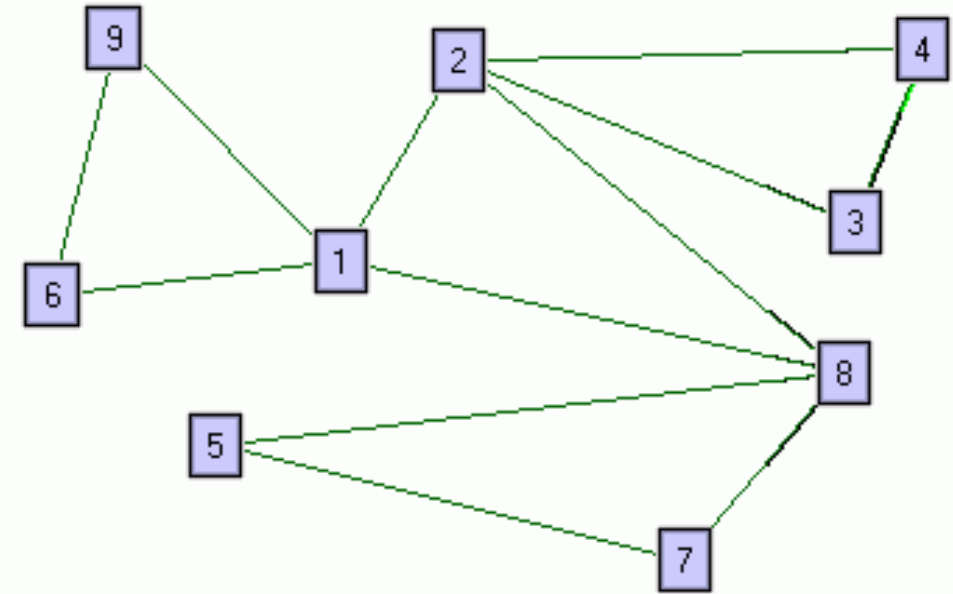
Does this graph have an Eulerian path?

Can you return to the starting node using the Eulerian path?



Solution

- Yes and yes.
- The first way to solve it would be with Euler's generalization.
- The second one would be using an algorithm that traverses the graph and checks if you can really do it.
- See <http://www.graph-magics.com/articles/euler.php>

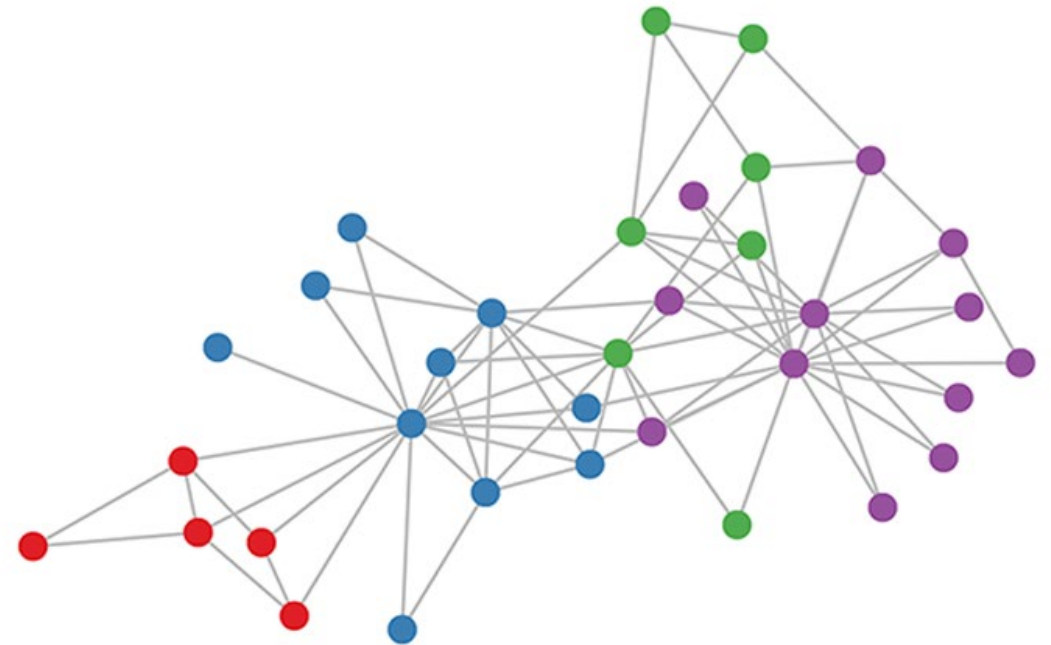


The beauty of graphs is that you can operate on them easily.

We can create algorithms to infer properties of systems represented as graphs, with many applications.

Applications (some of them)

1. Computer Science (PageRank, crawling)
2. Logistics (optimize package delivery)
3. Social sciences (rumor spreading)
4. Linguistics (natural language processing)
5. Modeling of neural networks
6. Chemistry (model compounds)
7. Firewalls, epidemic control
8. Computer Networks (shortest path routing)





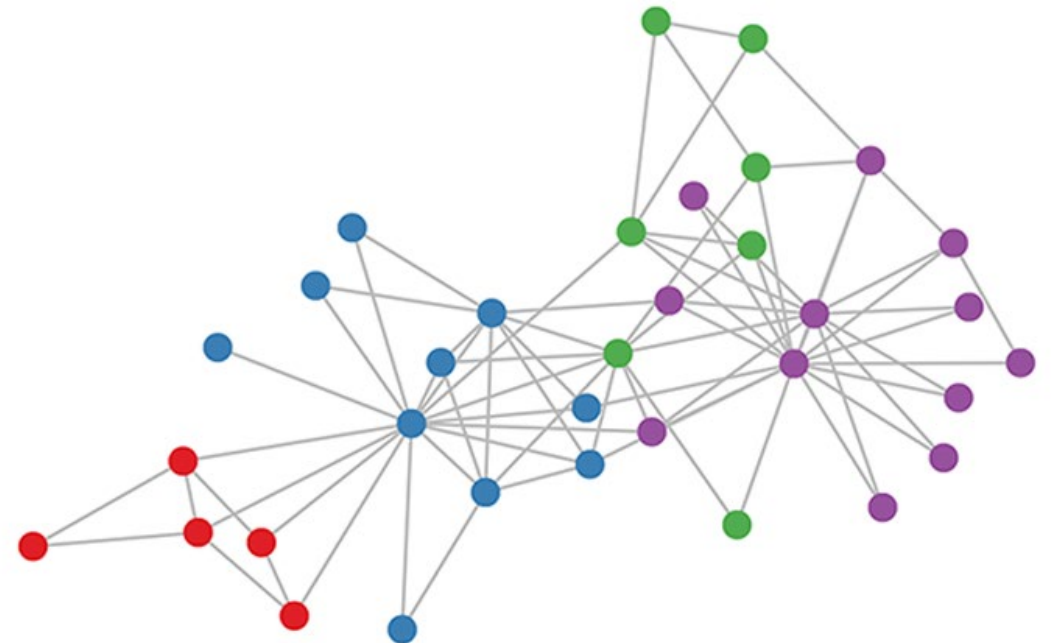
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Some useful algorithms for networking

Applications (some of them)

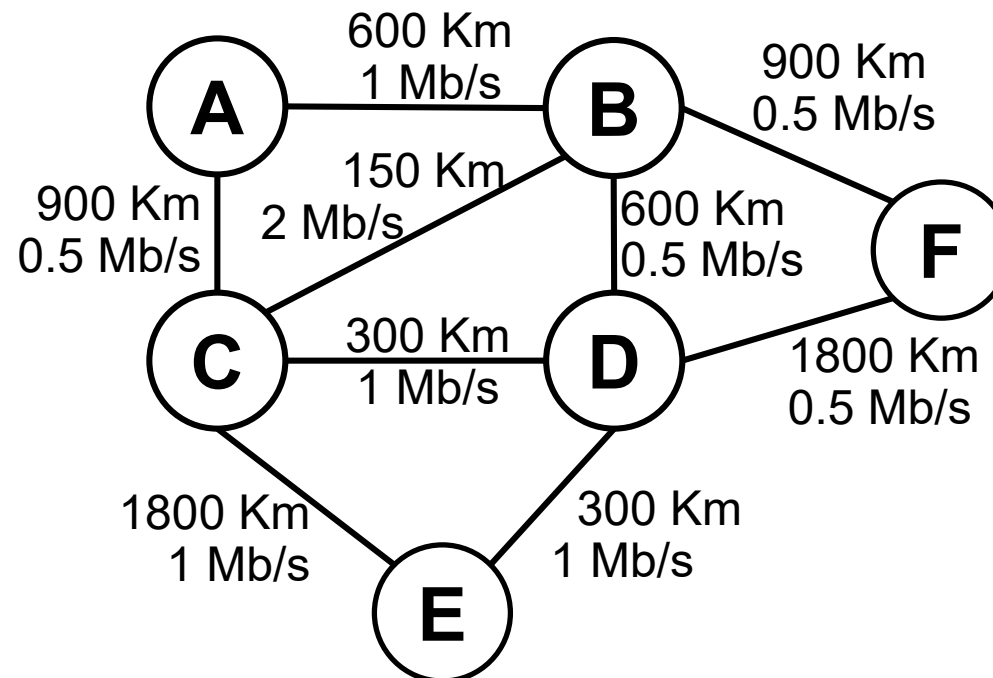
1. Computer Science (PageRank, crawling)
2. Logistics (optimize package delivery)
3. Social sciences (rumor spreading)
4. Linguistics (natural language processing)
5. Modeling of neural networks
6. Chemistry (model compounds)
7. Firewalls, epidemic control
8. **Computer Networks
(shortest path routing)**



Remember this graph?

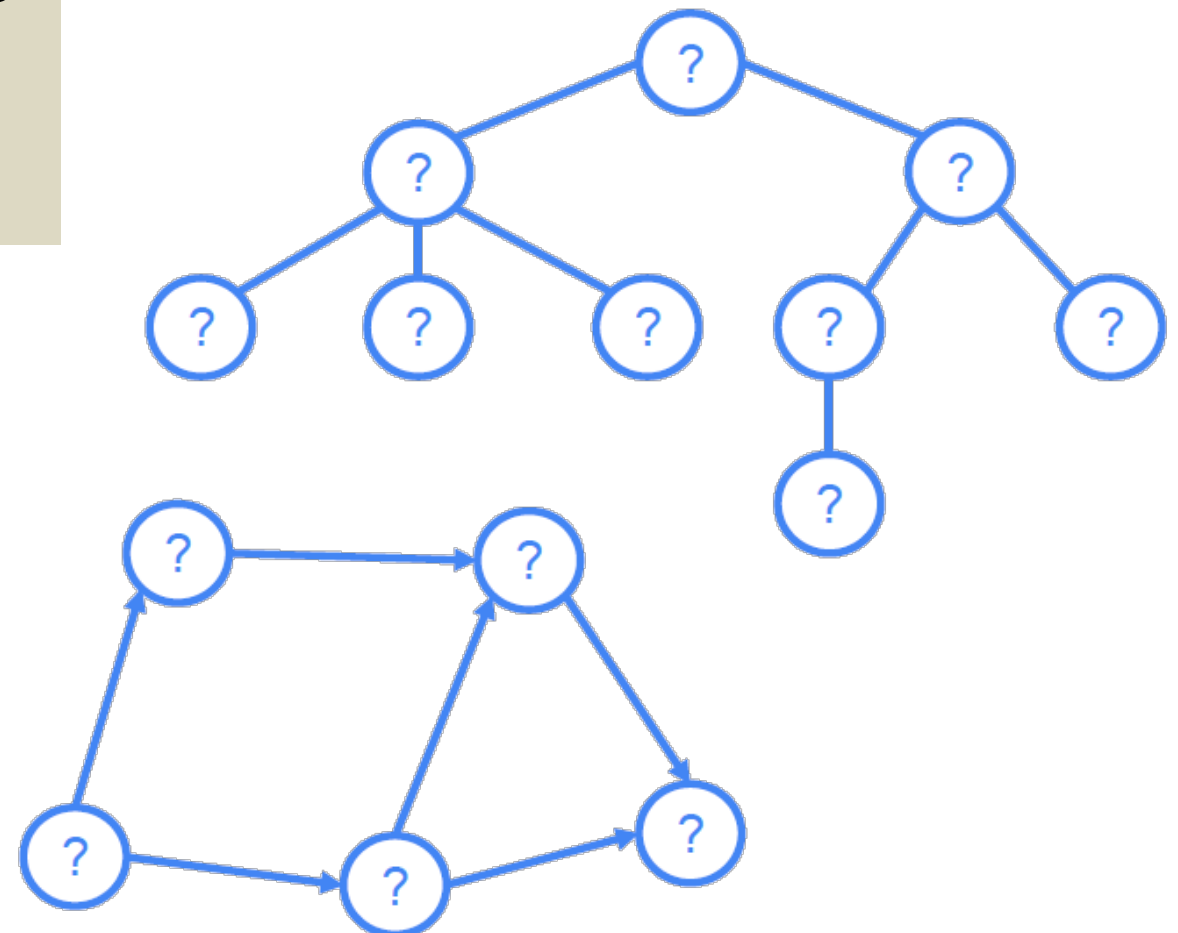
Consider several players connected through the graph below. Assuming a packet length of 1000 bits, and no queuing or processing delays, which is the best path:

- From A to F?
- From A to C?
- From A to E?



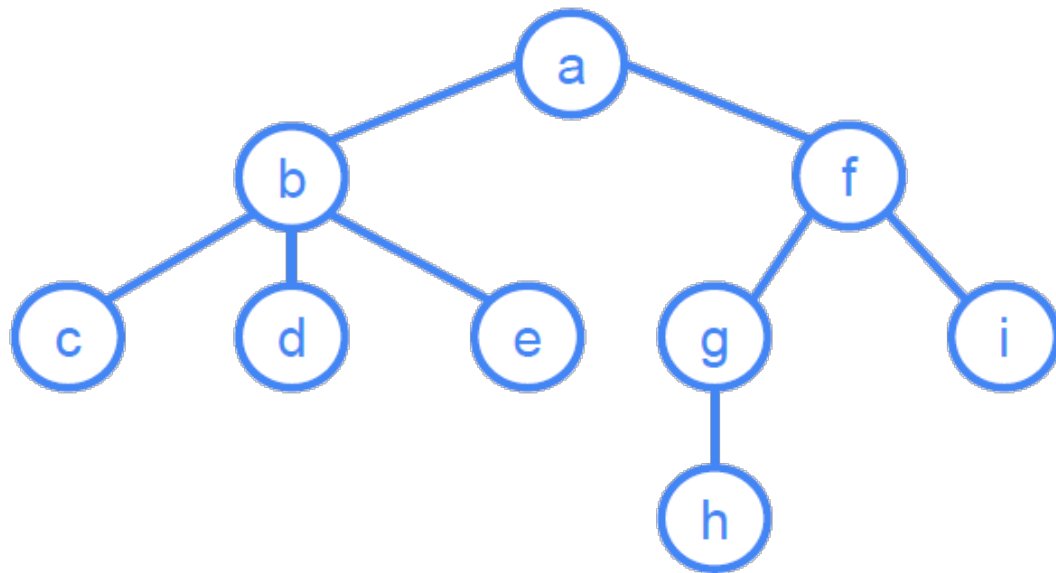
DFS (Depth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



DFS (Depth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



DFS Algorithm

Input: a graph G and an initial node v

Output: sequence of visited nodes

new stack S

new node $next$

mark v as visited

print(v)

add v to the stack S

while S is not empty **do**

$next \rightarrow$ unvisited adjacent node to the one in S head

 /* if there's not any unvisited adjacent, $next = NULL$ */

if ($next == NULL$)

 delete(S head node)

else aux has unvisited adjacent elements **do**

 mark $next$ as visited

 print($next$)

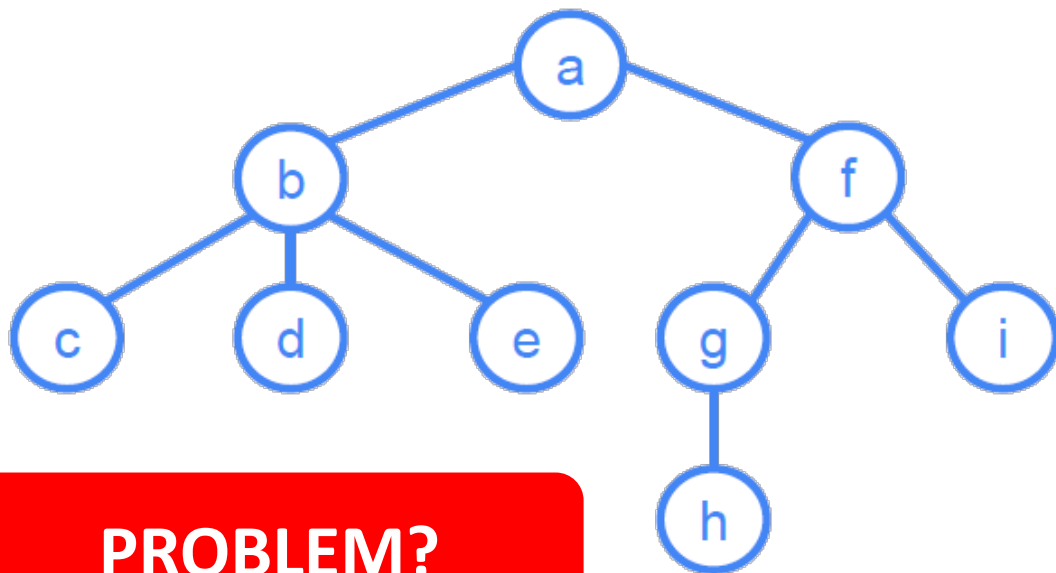
 add $next$ to S

end

end

DFS (Depth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



PROBLEM?

DFS Algorithm

Input: a graph G and an initial node v

Output: sequence of visited nodes

new stack S

new node $next$

mark v as visited

print(v)

add v to the stack S

while S is not empty **do**

$next \rightarrow$ unvisited adjacent node to the one in S head
/* if there's not any unvisited adjacent, $next = NULL$ */

if ($next == NULL$)

delete(S head node)

else aux has unvisited adjacent elements **do**

mark $next$ as visited

print($next$)

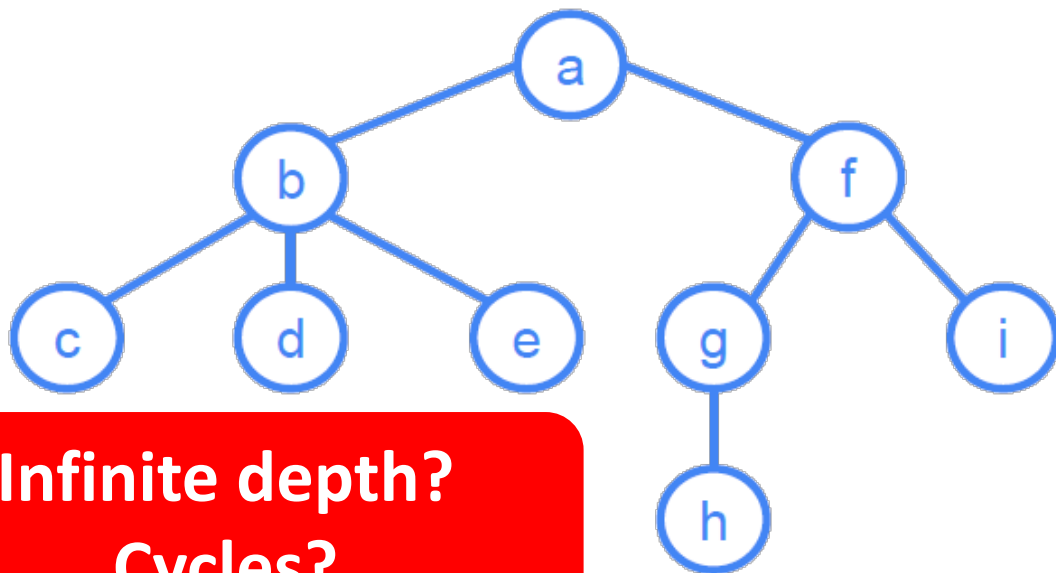
add $next$ to S

end

end

DFS (Depth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



Infinite depth?
Cycles?

DFS Algorithm

Input: a graph G and an initial node v

Output: sequence of visited nodes

new stack S

new node $next$

mark v as visited

print(v)

add v to the stack S

while S is not empty **do**

$next \rightarrow$ unvisited adjacent node to the one in S head
 /* if there's not any unvisited adjacent, $next = NULL$ */

if ($next == NULL$)

 delete(S head node)

else aux has unvisited adjacent elements **do**

 mark $next$ as visited

 print($next$)

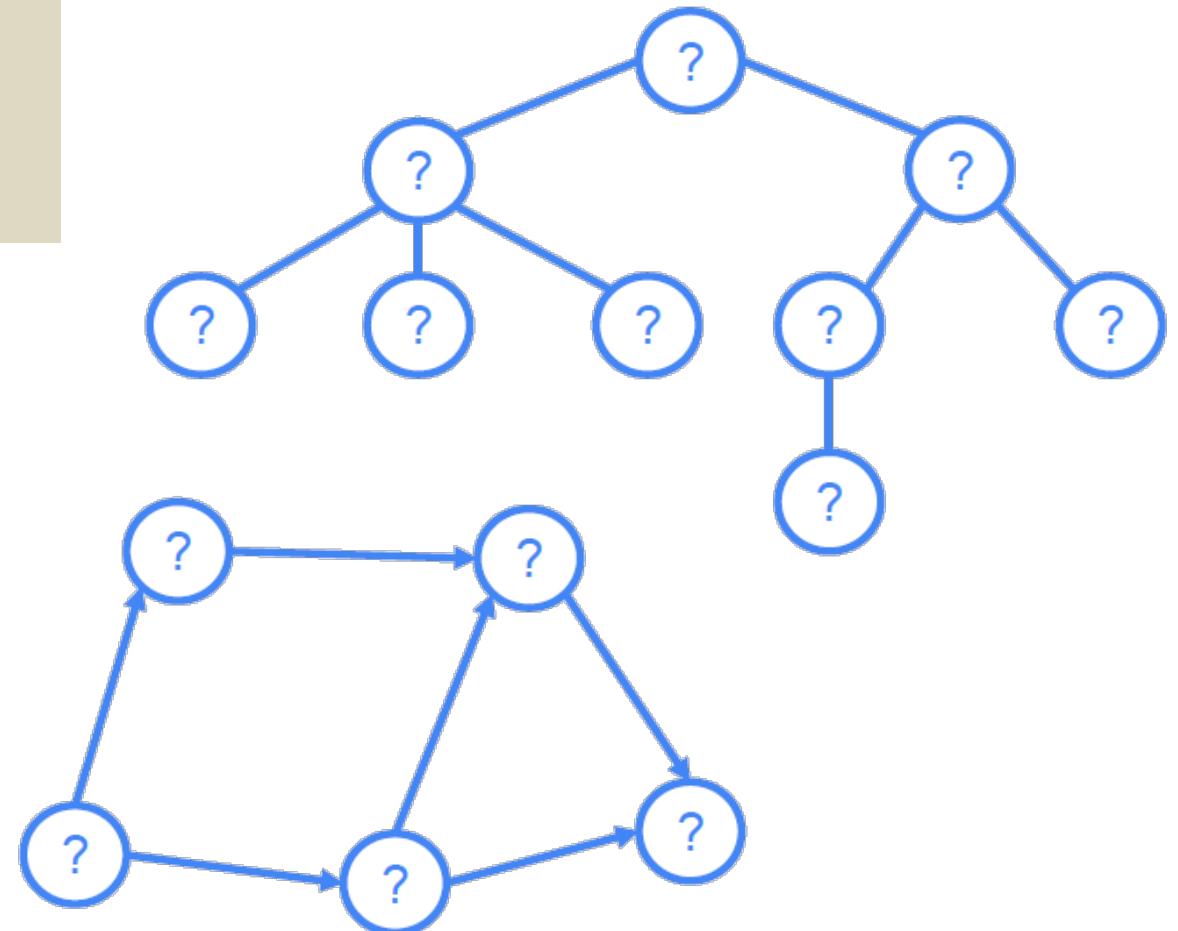
 add $next$ to S

end

end

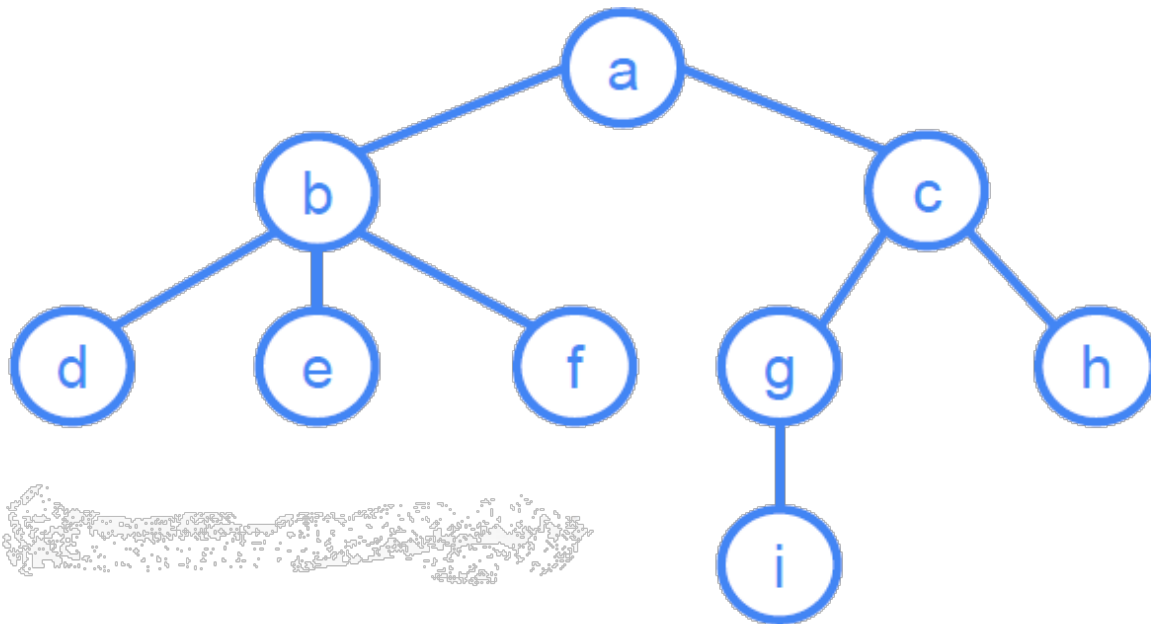
BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



BFS Algorithm

Input: a graph G and an initial node v

Output: sequence of visited nodes

new queue Q

mark v as visited

add v to the queue Q

new node aux

new node $next$

while Q is not empty **do**

$aux \rightarrow$ first element of Q

 print(aux)

 delete(first element of Q)

while aux has unvisited adjacent elements **do**

 mark adjacent(aux) as visited

 add adjacent(aux) to Q

end

end

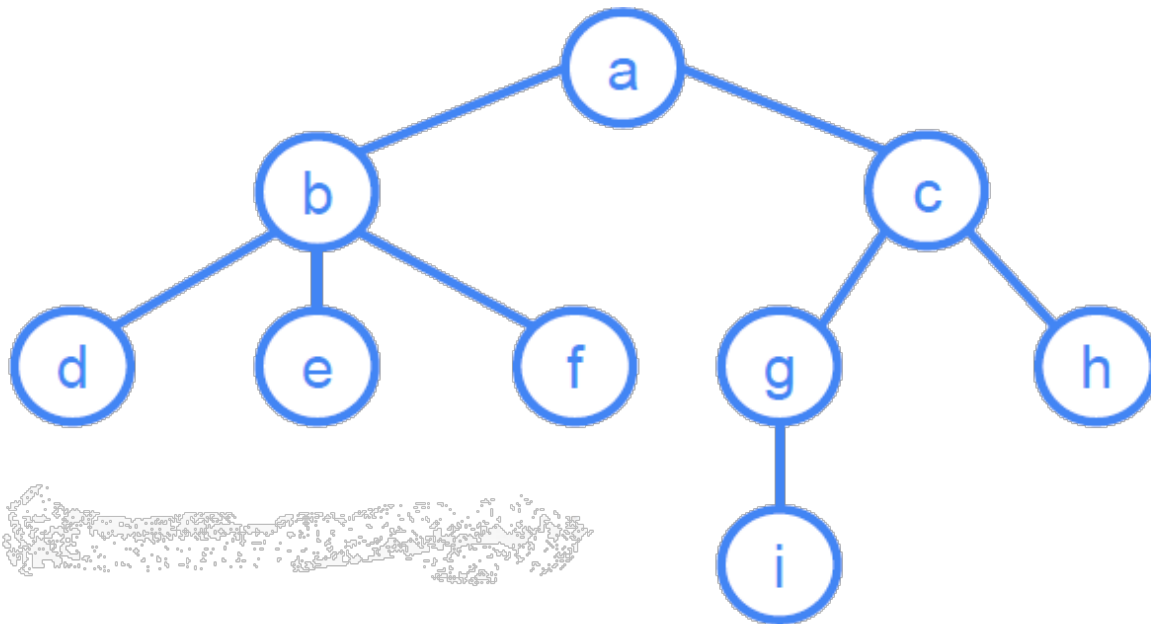
foreach node in G **do**

 mark it as not visited

end

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it



With the BFS algorithm we can know the distance of any node to the source node and, with that, know the shortest path from the source node to any other

This is NOT generalizable to DFS

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it

Application examples

1. Crawling and indexing of new webpages
2. Suggest contacts in social networks
3. Solving the Rubik cube
4. Protocol to find the shortest path between two nodes in a network.

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it

Application examples

4. Protocol to find the shortest path between two nodes in a network.

PROBLEM?

Each node in the network is a vertex, each link is an edge. With BFS you can make an exhaustive search until finding the shortest path to the destination. This was used in the *Routing Information Protocol (RIP)* for many years.

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it

Application examples

4. Protocol to find the shortest path between two nodes in a network.

Weighted graph?

Each node in the network is a vertex, each link is an edge. With BFS you can make an exhaustive search until finding the shortest path to the destination. This was used in the *Routing Information Protocol (RIP)* for many years.

Meet Professor Dijkstra



CITM

What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city. It is the algorithm for the shortest path, which I designed in about twenty minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a twenty minute invention. In fact, it was published in '59, three years late. The publication is still readable, it is, in fact, quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities. Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame.

Edsger Wybe Dijkstra
Rotterdam 1930 – Nuenen 2002

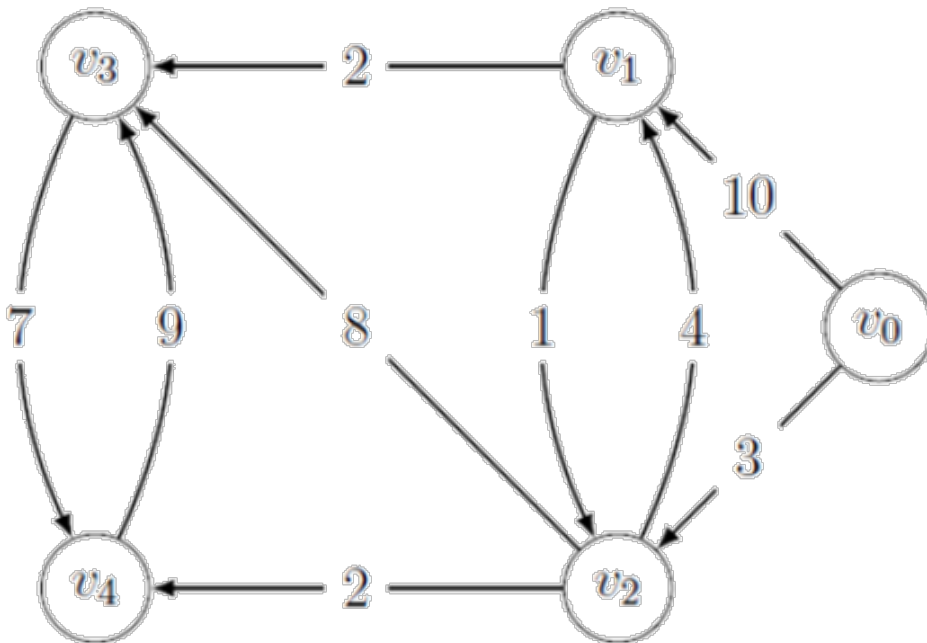
*Turing Prize 1972
ACM Influential Paper Award 2002
(now Dijkstra Prize)*



Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.



		Distàncies provisionals				
		v_0	v_1	v_2	v_3	v_4
Iteració	1	0	10	3	∞	∞
	2		7		11	5
	3		7		11	
	4				9	

Dijkstra's algorithm

Input: a weighted graph G and an initial node s .

$w(u, v) \in W$ is the weight of an edge between nodes u and v .

Output: minimum distance between s and the rest of nodes, minimum spanning tree

new dictionary $dist$

new dictionary Q

foreach node v in G **do**

$Q[v] = \infty$

$dist[v] = \infty$

end

$Q[s] = 0$

while Q is not empty **do**

$u = \min(Q)$

$dist[u] = Q[u]$

foreach node v adjacent to u **do**

if v is in Q **then**

if $Q[v] > Q[u] + w(u, v)$ **then**

$Q[v] = Q[u] + w(u, v)$

end

end

end

$delete(Q[u])$

end

$print(dist)$

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Another example

<https://www.youtube.com/watch?v=pVfj6mxhdMw>

Dijkstra's algorithm

Input: a weighted graph G and an initial node s .

$w(u, v) \in W$ is the weight of an edge between nodes u and v .

Output: minimum distance between s and the rest of nodes, minimum spanning tree

new dictionary $dist$

new dictionary Q

foreach node v in G **do**

$Q[v] = \infty$

$dist[v] = \infty$

end

$Q[s] = 0$

while Q is not empty **do**

$u = \min(Q)$

$dist[u] = Q[u]$

foreach node v adjacent to u **do**

if v is in Q **then**

if $Q[v] > Q[u] + w(u, v)$ **then**

$Q[v] = Q[u] + w(u, v)$

end

end

end

 delete($Q[u]$)

end

print($dist$)

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

1. GPS navigation
2. Currency exchange
3. Planning movements of a robot
4. Epidemiology
5. Routing (in networks)

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

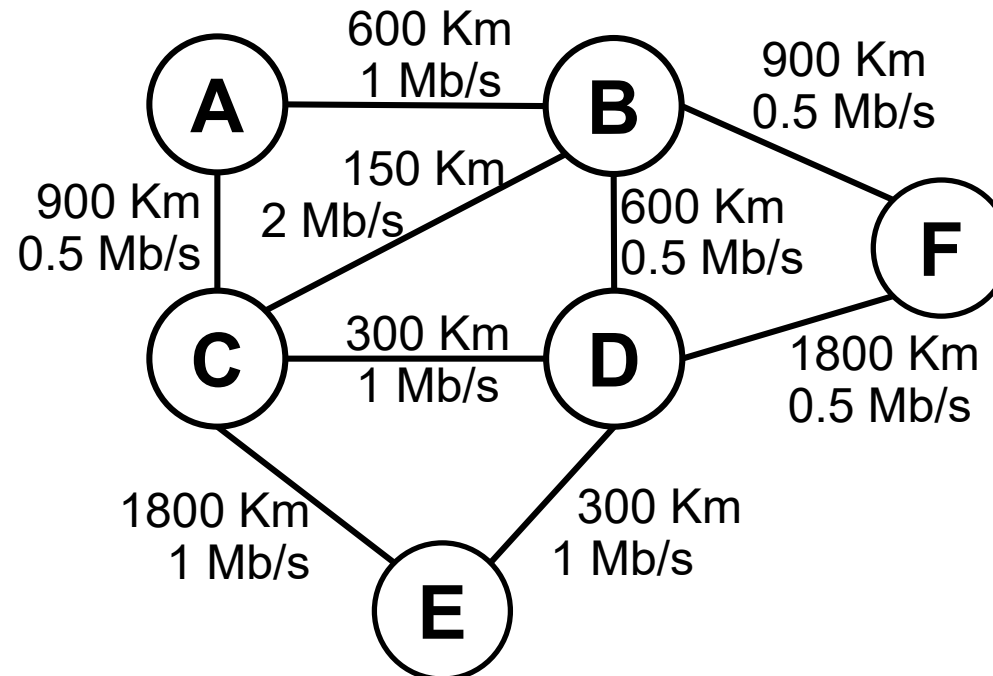
1. GPS navigation
2. Currency exchange
3. Planning movements of a robot
4. Epidemiology
5. **Routing (in networks)**

Best way to route packets to the destination node taking into consideration the state (weight) of each link.

Exercise – Pathfinding in a network

Consider several players connected through the graph below. Assuming a packet length of 1000 bits, and no queuing or processing delays, which is the best path:

- From A to F?
- From A to C?
- From A to E?

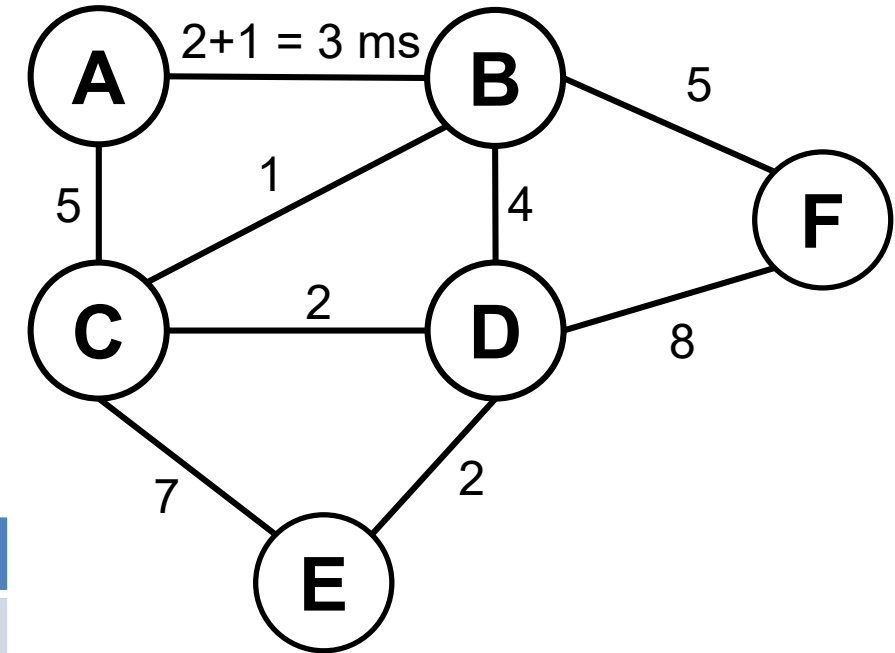


Exercise – Pathfinding in a network

We calculate the latency per link
(transmission and propagation)

Then we apply Dijkstra

	A	B	C	D	E	F	
1	-	<u>3</u>	5	∞	∞	∞	A->B
2	-	-	<u>4</u>	7	∞	8	<u>A->B->C</u>
3	-	-	-	<u>6</u>	11	8	A->B->C->D
4	-	-	-	-	<u>8</u>	8	<u>A->B->C->D->E</u>
5	-	-	-	-	-	<u>8</u>	<u>A->B->F</u>



Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Widely used protocols such as OSPF or ISIS use Dijkstra's algorithm to determine the shortest path between any two nodes mostly backbone and large enterprise networks.

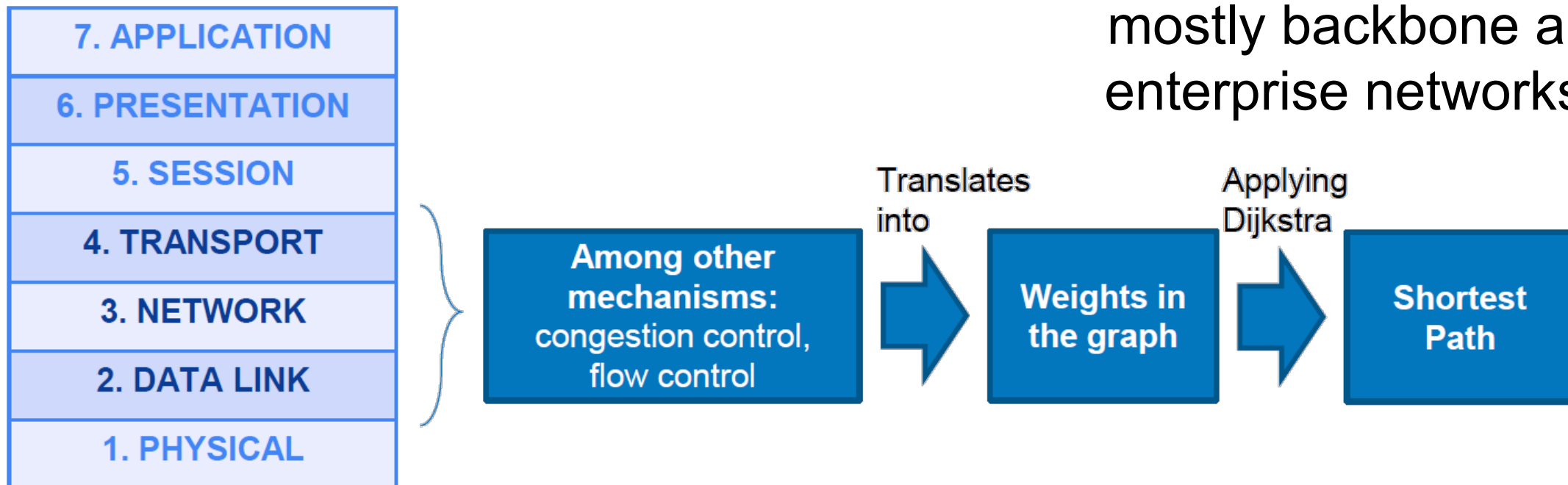
Implementations [\[edit \]](#)

- [Allied Telesis](#) implements OSPFv2 & OSPFv3 in Allied Ware Plus (AW+)
- [Arista Networks](#) implements OSPFv2 and OSPFv3
- [BIRD](#) implements both OSPFv2 and OSPFv3
- [Cisco IOS](#)
- [Cisco Meraki](#)
- [D-Link](#) implements OSPFv2 on Unified Services Router.
- [Dell's FTOS](#) implements OSPFv2 and OSPFv3
- [ExtremeXOS](#)
- [GNU Zebra](#), a GPL routing suite for [Unix-like](#) systems supporting OSPF
- [Juniper Junos](#)
- [Mikrotik RouterOS](#)
- [NetWare](#) implements OSPF in its Multi Protocol Routing module.
- [OpenBSD](#) includes [OpenSPFD](#), an OSPFv2 implementation.
- [Quagga](#), a fork of [GNU Zebra](#) for [Unix-like](#) systems
- [XORP](#), a routing suite implementing RFC2328 (OSPFv2) and RFC2740 (OSPFv3) for both IPv4 and IPv6
- [Windows NT 4.0 Server](#), [Windows 2000 Server](#) and [Windows Server 2003](#) implement OSPFv2 in the [Routing and Remote Access Service](#), although the functionality was removed in [Windows Server 2008](#).

Dijkstra's Algorithm

Developed by Dijkstra in 1956.
Finds the shortest path between two nodes in a weighted graph.

Widely used protocols such as OSPF or ISIS use Dijkstra's algorithm to determine the shortest path between any two nodes mostly backbone and large enterprise networks. **How?**





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Introduction

1.3 Graph Theory

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

For further reading

Aric Hagberg, Dan Schult, Pieter Swart. “*NetworkX Reference*”. <https://networkx.github.io/documentation/stable/>

NetworkX: “*Software for complex networks*”. Github
<http://networkx.github.io/>

Aniol Garcia “*Els grafs: xarxes, camins i connexions. De la matemàtica discreta a la realitat*”. Premi Pointcaré 2017

Journal of Graph Algorithms and Applications. ISSN: 1526-1719. <http://jgaa.info>



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Extra material



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Graph Types (II)

Graph types (II)



CITM

- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - Complete
 - Bipartite
 - Planar
 - Tree
 - Hamiltonian
 - Eulerian

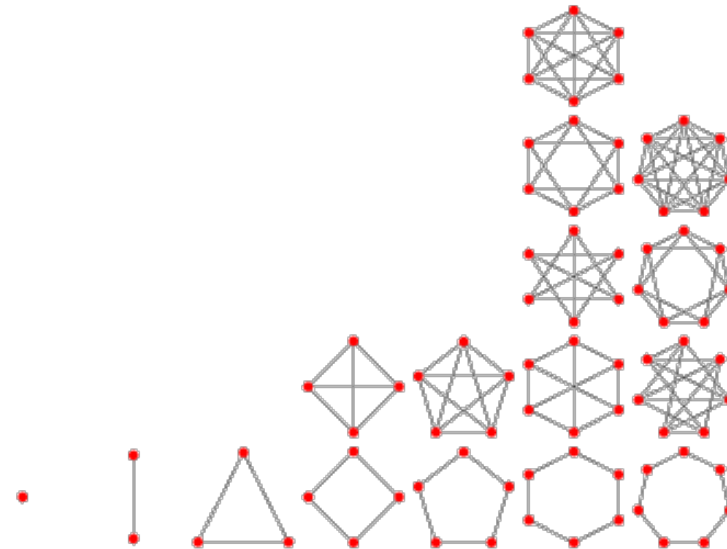
Do you think the
graph of a
network like the
Internet can be
of these types?



Graph types (II)

- The second distinction between types of graphs comes from particular arrangements
 - **Regular**
 - Complete
 - Bipartite
 - Planar
 - Tree
 - Hamiltonian
 - Eulerian

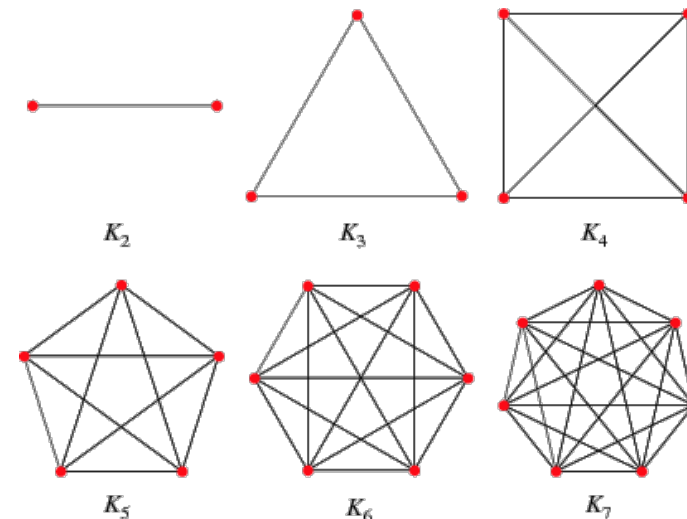
A graph is regular if all vertices have the same degree



Graph types (II)

- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - **Complete**
 - Bipartite
 - Planar
 - Tree
 - Hamiltonian
 - Eulerian

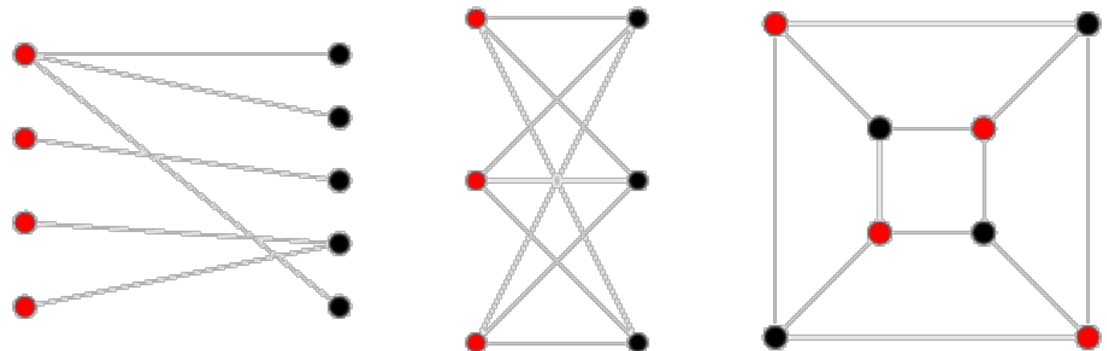
A graph is complete if each pair of vertices is connected by one edge



Graph types (II)

- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - Complete
 - **Bipartite**
 - Planar
 - Tree
 - Hamiltonian
 - Eulerian

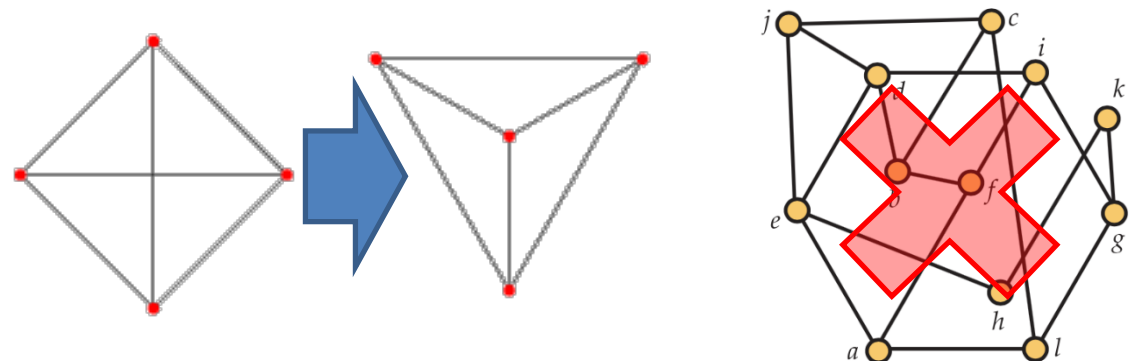
A graph is bipartite if all vertices can be grouped into two disjoint and independent units



Graph types (II)

- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - Complete
 - Bipartite
 - **Planar**
 - Tree
 - Hamiltonian
 - Eulerian

A graph is planar if it can be embedded in a plane, this is, whose edges can be drawn without crossing

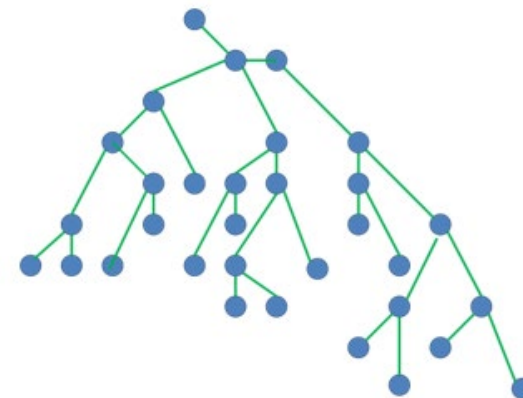


Graph types (II)



- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - Complete
 - Bipartite
 - Planar
 - **Tree**
 - Hamiltonian
 - Eulerian

A tree is a graph without cycles, this is, where any two vertices are connected through one path at most



- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - Complete
 - Bipartite
 - Planar
 - Tree
 - **Hamiltonian**
 - Eulerian

A graph is Hamiltonian if there is a path that goes through all the vertices only once

- The second distinction between types of graphs comes from particular arrangements
 - Regular
 - Complete
 - Bipartite
 - Planar
 - Tree
 - Hamiltonian
 - **Eulerian**

A graph is Eulerian if there is a path that goes through all the edges only once

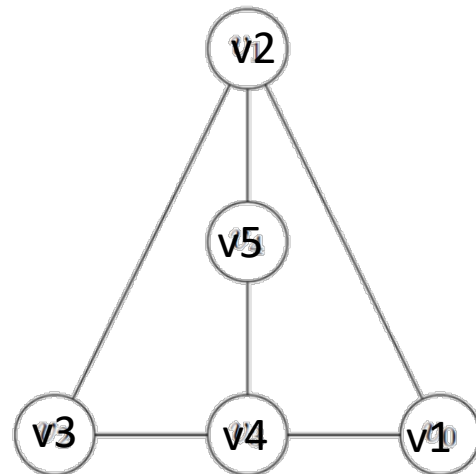


UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Graph Data Structures

- The first step to operate over graphs is to know how to store the graphs in memory. Two options:
 - **Adjacency Matrix:** a $|V|$ -to- $|V|$ matrix where a value of X (or 0) in the position (i,j) means that there is (not) an edge of weight X between vertices i and j . Value can be 1 if graph is not weighted. Symmetric if the graph is undirected, probably asymmetric if directed.



	v_1	v_2	v_3	v_4	v_5
v_1	0	1	0	1	0
v_2	1	0	1	0	1
v_3	0	1	0	1	0
v_4	1	0	1	0	1
v_5	0	1	0	1	0

- The first step to operate over graphs is to know how to store the graphs in memory. Two options:
 - **Adjacency Matrix:** a $|V|$ -to- $|V|$ matrix where a value of X (or 0) in the position (i,j) means that there is (not) an edge of weight X between vertices i and j . Value can be 1 if graph is not weighted. Symmetric if the graph is undirected, probably asymmetric if directed.



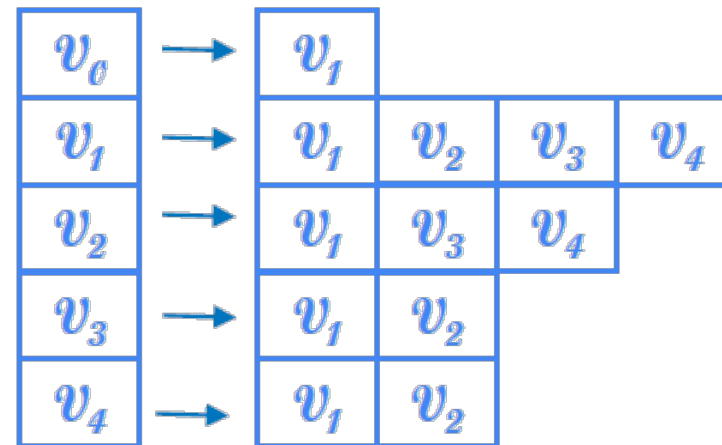
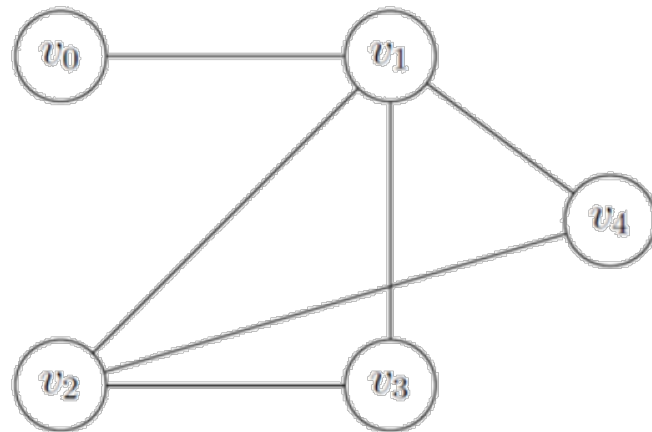
	v_1	v_2	v_3	v_4	v_5
--	-------	-------	-------	-------	-------

Pro: easy to walk through since the dimensions are fixed in a single structure.
Con: large unnecessary memory footprint for large graphs with sparse connectivity.

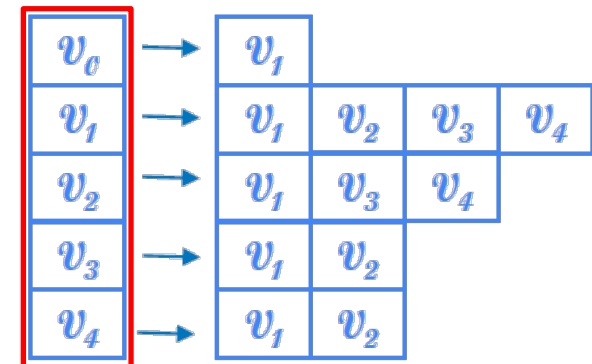


v_5	0	1	0	1	0
-------	---	---	---	---	---

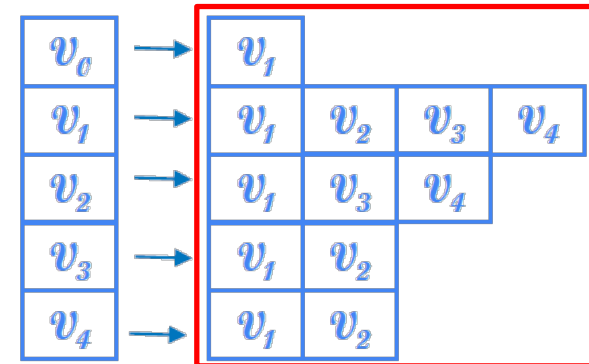
- The first step to operate over graphs is to know how to store the graphs in memory. Two options:
 - Adjacency Matrix
 - **Adjacency List:** a main list containing the vertices of the graph, pointing to $|V|$ adjacency lists containing the connected vertices.



- The first step to operate over graphs is to know how to store the graphs in memory. Two options:
 - Adjacency Matrix
 - **Adjacency List:** the main list has to contain
 - An identifier of the node or vector
 - Information specific to the algorithm to be implemented:
 - Node or vector already visited: Yes/No
 - Array of ids of the explorations in which it has been included



- The first step to operate over graphs is to know how to store the graphs in memory. Two options:
 - Adjacency Matrix
 - **Adjacency List:** the adjacency lists have to contain
 - An identifier of the node or vector
 - Information specific to the algorithm to be implemented:
 - Weigh of the link: cost, distance, ...
 - Node or vector already visited: Yes/No
 - Array of ids of the explorations in which it has been included





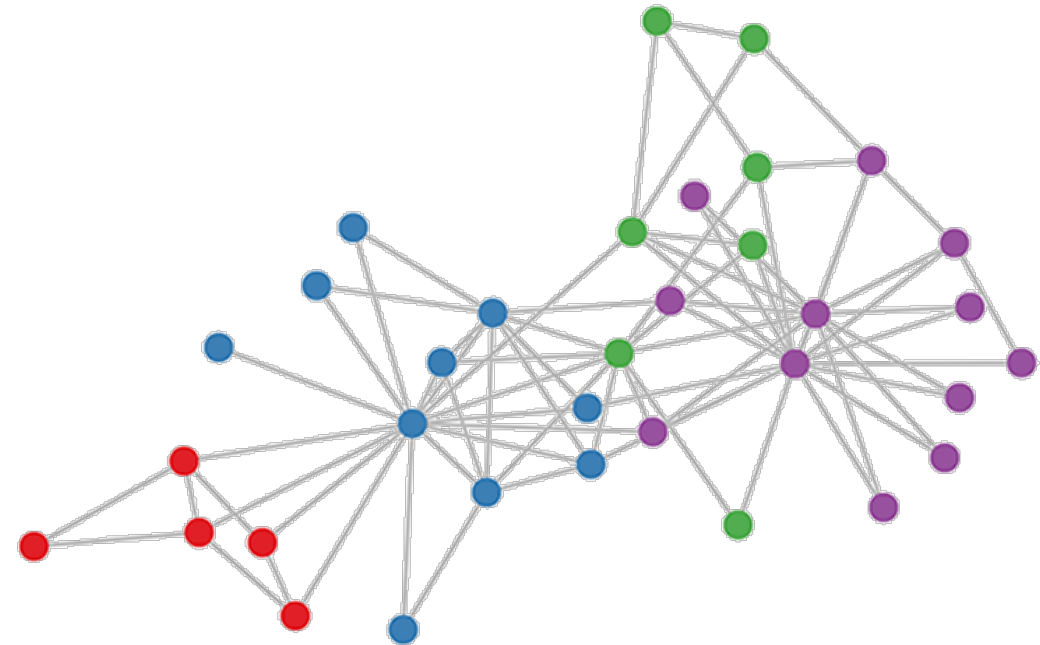
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Graph algorithms

Some graph algorithms

- The Hamiltonian path
- Maximum flow
- Maximum flow with minimum cost
- Minimum cut
- Maximal independent set
- Complete subgraphs
- Optimal graph coloring



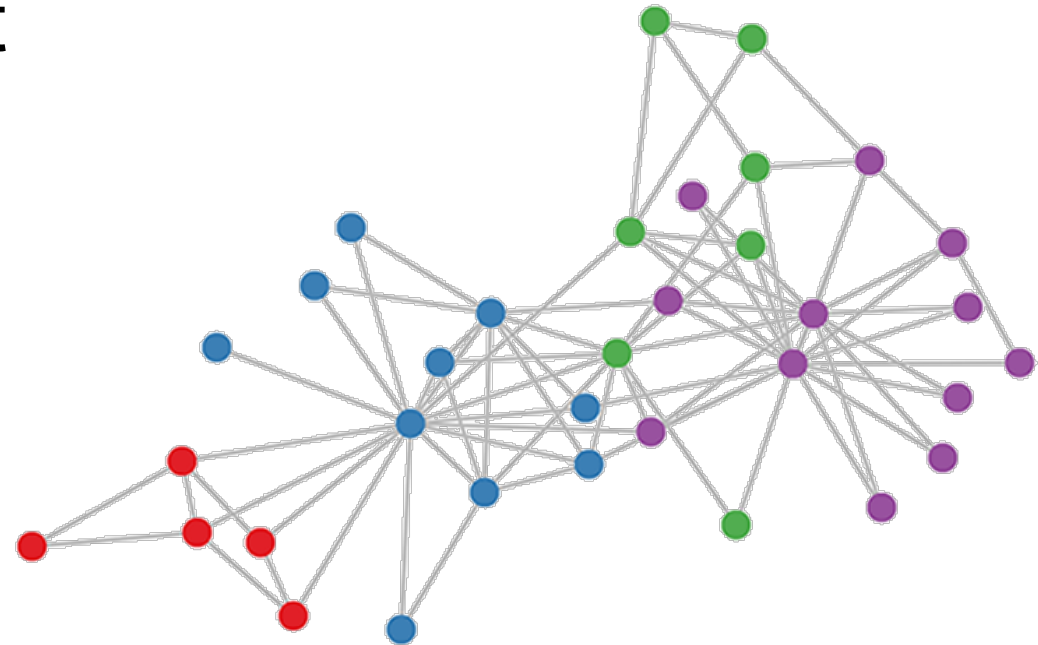
Some graph algorithms

- **The Hamiltonian path**

Finds a path that goes through all nodes only once

- Maximum flow
- Maximum flow with minimum cost
- Minimum cut
- Maximal independent set
- Complete subgraphs
- Optimal graph coloring

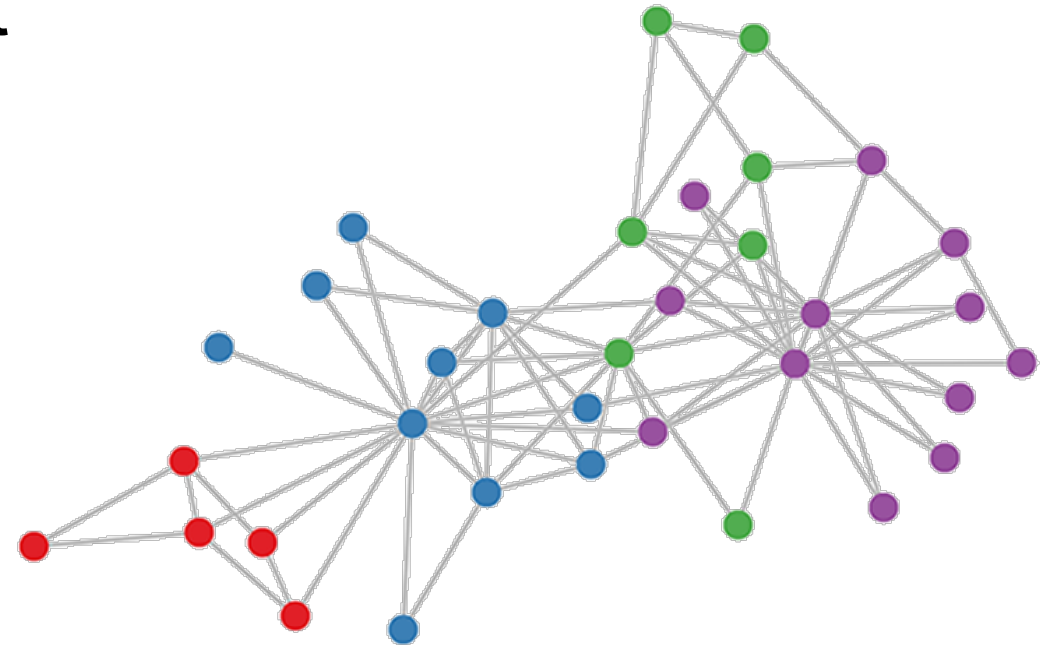
Logistics: create a school trip that goes through all students houses and goes back to the school.



Some graph algorithms

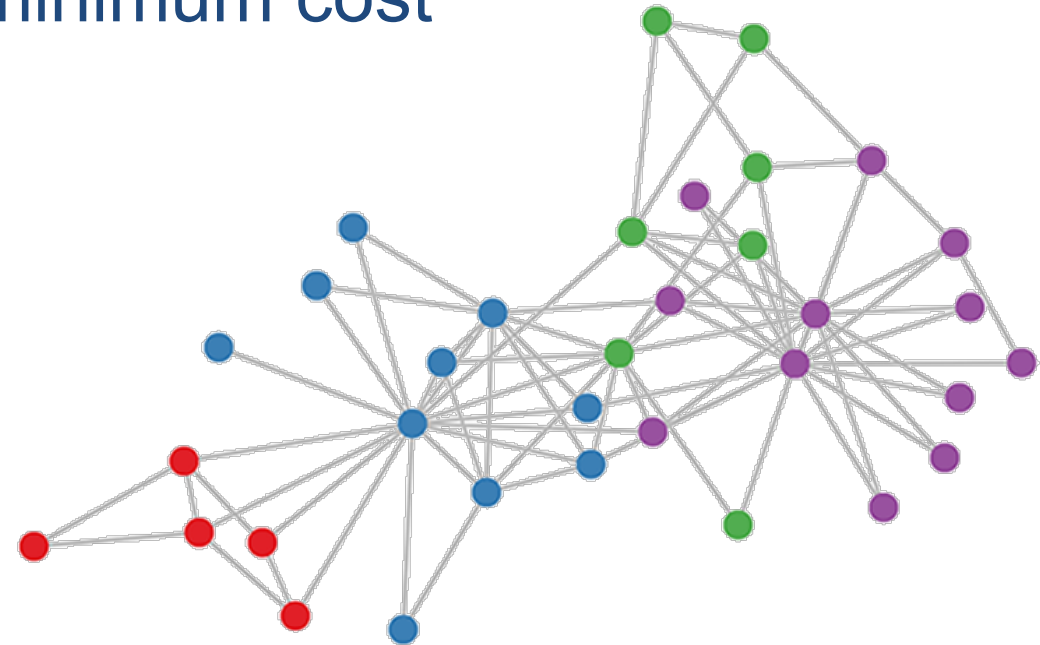
- The Hamiltonian path
- **Maximum flow**
Find the path that allow the maximum flow (e.g. Water) between two nodes
- Maximum flow with minimum cost
- Minimum cut
- Maximal independent set
- Complete subgraphs
- Optimal graph coloring

Epidemy control: find the places where a disease or fire is most likely to propagate.



Some graph algorithms

- The Hamiltonian path
- Maximum flow
- **Maximum flow with minimum cost**
Find the path that allow the maximum flow
(e.g. Water) between two nodes with minimum cost
- Minimum cut
- Maximal independent set
- Complete subgraphs
- Optimal graph coloring



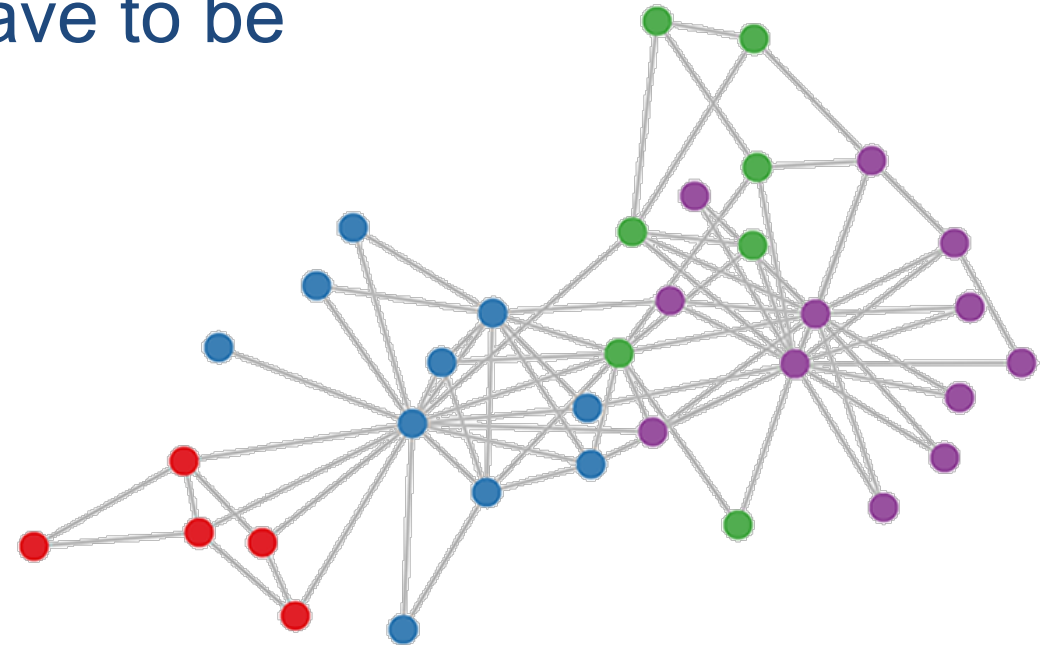
Some graph algorithms

- The Hamiltonian path
- Maximum flow
- Maximum flow with minimum cost
- **Minimum cut**

Find the lowest number of links that have to be eliminated to isolate two given nodes

- Maximal independent set
- Complete subgraphs
- Optimal graph coloring

Networks: to reinforce paths that, if broken, can leave customers without Internet services

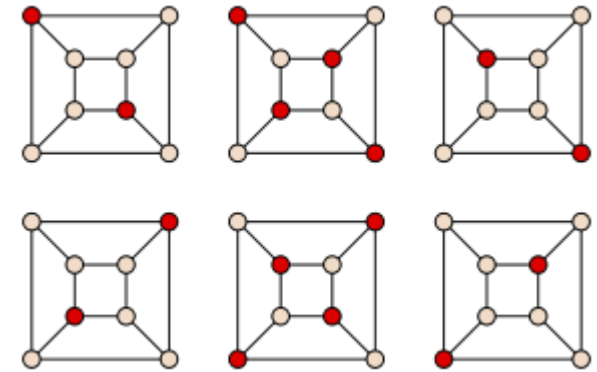


Some graph algorithms



CITM

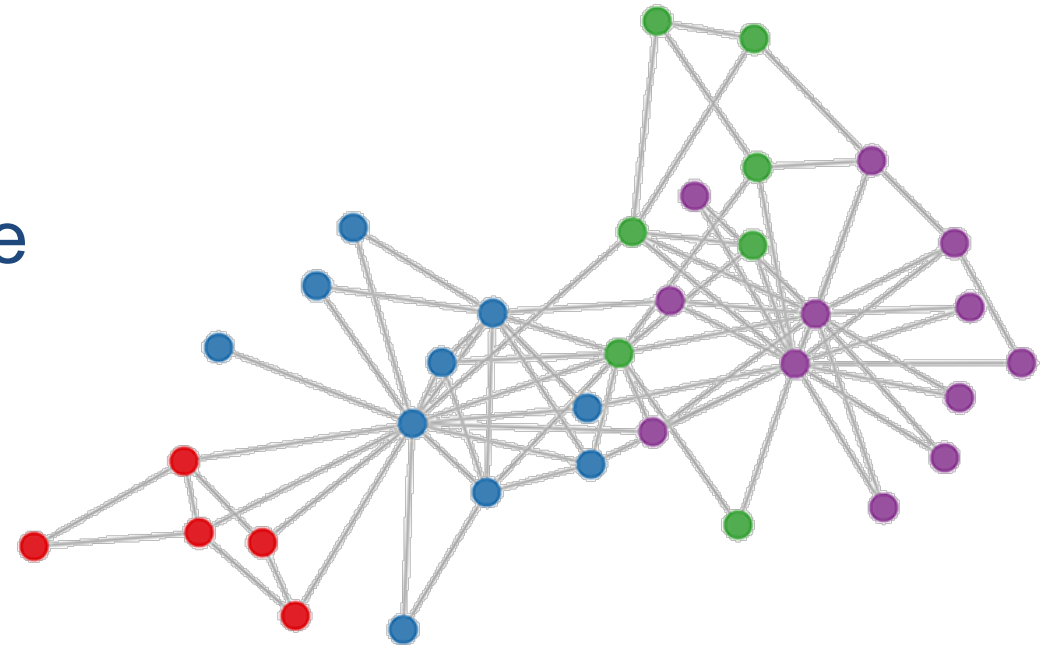
- The Hamiltonian path
- Maximum flow
- Maximum flow with minimum cost
- Minimum cut
- **Maximal independent set**
Find the largest subset of nodes, all of which are not directly connected to each other
- Complete subgraphs
- Optimal graph coloring



Some graph algorithms

- The Hamiltonian path
- Maximum flow
- Maximum flow with minimum cost
- Minimum cut
- Maximal independent set
- **Complete subgraphs – clique**
Find the largest subset of nodes where all nodes are connected to each other
- Optimal graph coloring

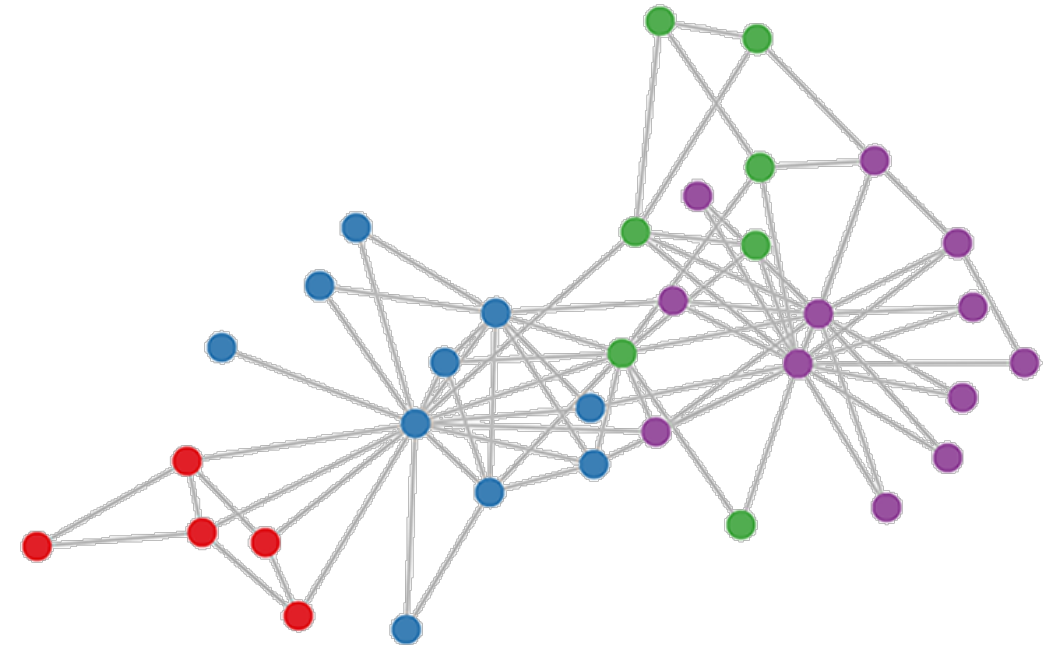
Cybersecurity: Fraud detection using bank transfers and other data



Some graph algorithms

- The Hamiltonian path
- Maximum flow
- Maximum flow with minimum cost
- Minimum cut
- Maximal independent set
- Complete subgraphs
- **Optimal graph coloring**
Assign a color to each node avoiding that adjacent nodes have the same color, with the minimum number of colors

Networking: Assign channels to base stations wireless networks to minimize cost





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Applications of BFS and Dijkstra

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it

Application examples

- 1. Crawling and indexing of new webpages**
2. Suggest contacts in social networks
3. Solving the Rubik cube

With BFS you can go through the entire Internet. Treat each webpage as a vertex and each hyperlink as an edge of a graph. Then, each unindexed webpage that is linked by an indexed page will be found by the algorithm

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it

Application examples

1. Crawling and indexing of new webpages
2. **Suggest contacts in social networks**
3. Solving the Rubik cube

Each person is a vertex, each relationship is an edge. With BFS you can search friends of the friends of a person (vertex at distance 2) that may be actually friends of that person. The more distance-2 paths two vertices share, the more likely is that they will know each other

BFS (Breadth First Search)

Algorithm used to examine the structure of a graph or to systematically explore it

Application examples

1. Crawling and indexing of new webpages
2. Suggest contacts in social networks
3. **Solving the Rubik cube**

Generating a graph where each node is a single cube state and the links are single face movements, given any initial state, BFS finds the shortest path to completion

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

1. **GPS navigation**
2. Currency exchange
3. Planning movements of a robot
4. Epidemiology
5. Routing (in networks)

It is used to find the shortest path between any two points in a map. The edges are streets and roads, the vertices are the intersections and the weights are the distances (or time required between the points).

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

1. GPS navigation
- 2. Currency exchange**
3. Planning movements of a robot
4. Epidemiology
5. Routing (in networks)

The best way to exchange currencies. Here vertices are the currencies, the edges are the transactions and the weights are the exchange rates.

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

1. GPS navigation
2. Currency exchange
- 3. Planning movements of a robot**
4. Epidemiology
5. Routing (in networks)

Each vertex is a position in the space. Filling the space free of obstacles with intermediate vertices, and executing the algorithm to the resulting graph, we obtain the optimal way to reach the target position.

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

1. GPS navigation
2. Currency exchange
3. Planning movements of a robot
- 4. Epidemiology**
5. Routing (in networks)

We can model group of people, their relations (close acquaintances), and their pathologies. It is easy to see who has the most changes of falling ill.



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Dijkstra can fail

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Application examples

1. GPS navigation
2. Currency exchange
3. Planning movements of a robot
4. Epidemiology
5. Routing (in networks)

PROBLEM?

Dijkstra's algorithm

Input: a weighted graph G and an initial node s .

$w(u, v) \in W$ is the weight of an edge between nodes u and v .

Output: minimum distance between s and the rest of nodes, minimum spanning tree

new dictionary $dist$

new dictionary Q

foreach node v in G **do**

$Q[v] = \infty$

$dist[v] = \infty$

end

$Q[s] = 0$

while Q is not empty **do**

$u = \min(Q)$

$dist[u] = Q[u]$

foreach node v adjacent to u **do**

if v is in Q **then**

if $Q[v] > Q[u] + w(u, v)$ **then**

$Q[v] = Q[u] + w(u, v)$

end

end

end

 delete($Q[u]$)

end

print($dist$)

Dijkstra's Algorithm

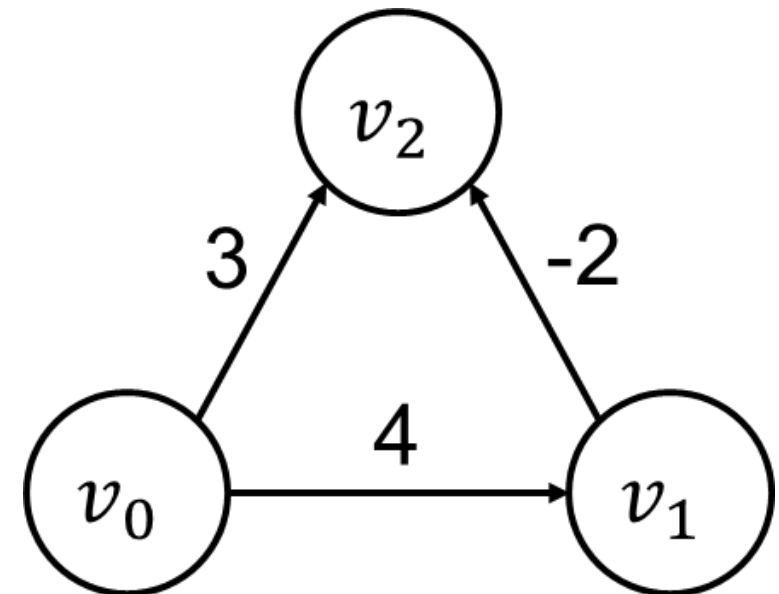
Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Dijkstra does not work when the weights can be negative.

This simple graph proves it.

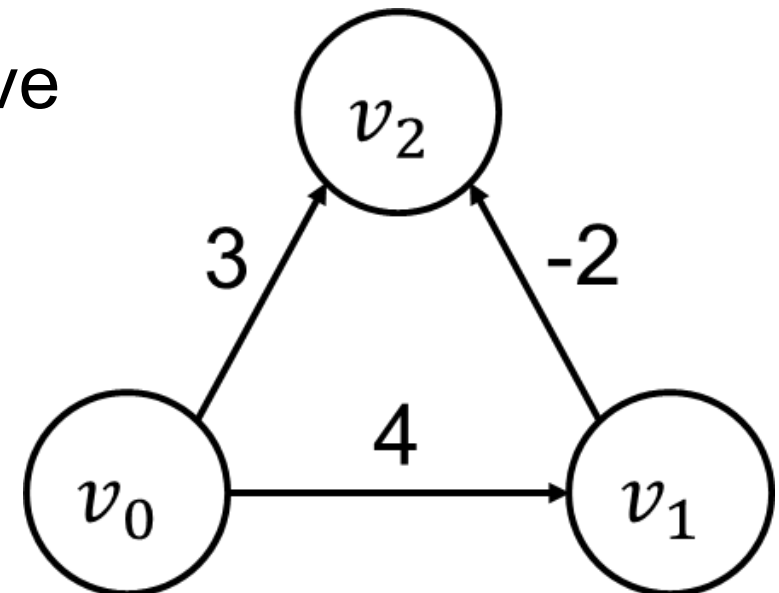
Use Dijkstra to find the shortest path from v_0 to v_2 .



Other advanced algorithms for the shortest-path problem:

- If there are negative weights: Bellman-Ford
Similar to Dijkstra, but checks all edges for all nodes to account for possible negative weights

<https://www.youtube.com/watch?v=obWXjtg0L64>



Other advanced algorithms for the shortest-path problem:

- If there are negative weights: Bellman-Ford
- If you want to go faster using heuristics: A^* (best-first!)
- If you want to know shortest paths for all sources: Floyd-Warshall
- If you want all-source, but faster in sparse graph: Johnson's algorithm
- If your weights are stochastic (probabilities): Viterbi algorithm



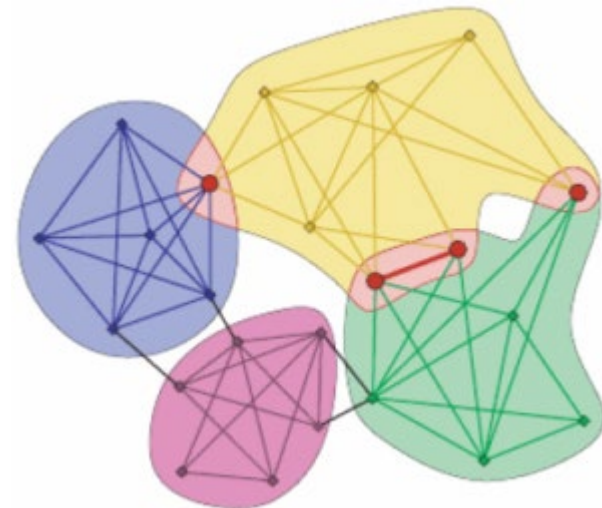
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

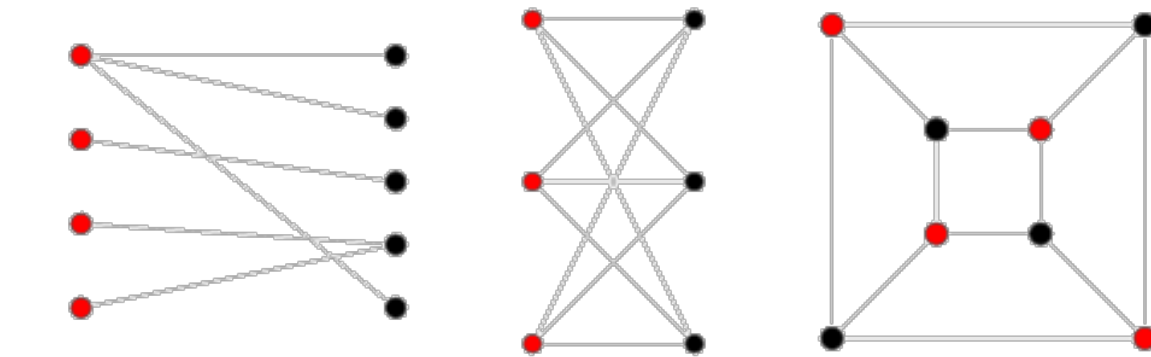
Applications of graph algorithms

Clique – Community detection

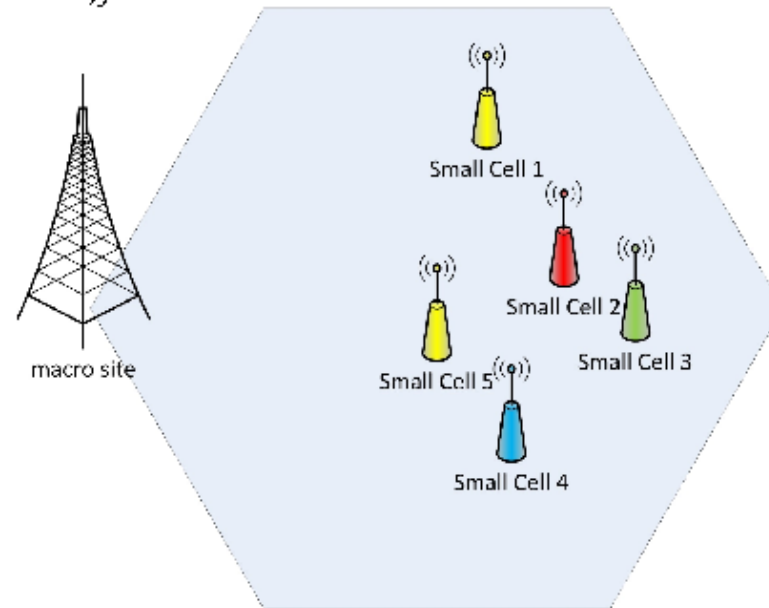
- In social networks, to detect close groups of friends.
- Customer segmentation
- Astroturfing or nepotism in politics
- Fraud detection



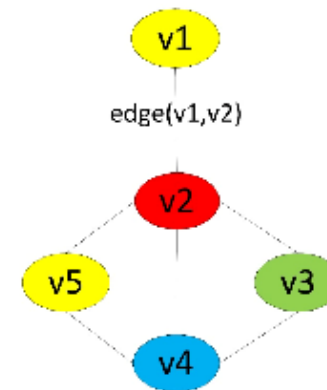
Graph coloring in wireless cellular networks



that $a_{i,j} \leq a_{reuse}$.



Sketch of System Layout



Corresponding Interference Graph G

Fig. 1 Construction and Coloring of interference graph

Discussion: Identify a functionality in a videogame or in networks where the graph algorithms just described may be applicable.

